# A Comparative Study on the Suitability of Cloud Simulators for Estimating Host-Level Energy Usage of Microservice Systems

Meret Baumgartner

*Vrije Universiteit Amsterdam, the Netherlands*

Max Karsten

*Vrije Universiteit Amsterdam, the Netherlands*

Ivano Malavolta

*Vrije Universiteit Amsterdam, the Netherlands*

**Abstract**

*Context.* With the promise of flexibility and scalability they provide, microservices have become widespread in industry and are often deployed in data centres, contributing to the cloud's large energy footprint. In order to reduce the energy usage of microservice systems and develop strategies to decrease resource consumption overall, simulation can be an important tool, but little research has been done into assessing cloud simulators' energy simulation capabilities in the context of microservice systems.
*Goal.* In this study, we analyze and compare five cloud simulators in terms of their host-level energy simulation capabilities as well as their suitability to simulating microservice systems.
*Method.* We test the accuracy of the simulators by comparing their simulated host-level energy usage values to collected real-world measures for two microservice applications across different scenarios and userloads. Furthermore, we systematically analyze the energy-related capabilities and user-related features of the simulators, such as customizability.
*Results.* All five simulators take on average less than 4 seconds per simulation to run. *SimGrid* is on average the fastest with a mean runtime of 0.05. *OpenDC* is the simulator that simulates the energy usage with the highest accuracy with a maximum mean relative error of 0.41 of the total energy usage, followed by *SimGrid* at 0.42. From a user perspective OpenDC is the simplest to start using, while the *DISSECT-CF* simulator is the most customizable and enables easy loading of workload traces. *SimGrid* has the most extensive energy-related capabilities.
*Conclusions.* Our results show that, in general, *OpenDC* is best suited for modelling host-level energy consumption of microservice systems out of the selected simulators. For specific contexts, we have provided descriptions and properties of the simulators that can enable potential users to make informed decisions on what simulator to use for their circumstances.

## 1. Introduction

Data centres consumed approximately 415 TWH, or 1.5% of global energy consumption in 2024, and this is expected to more than double by 2030 [1]. Cloud computing happens largely in data centers and significantly contributes to this number, which does not even take the water use [2] into account. The same goes for other far-reaching environmental impacts that remain obscured due to the largely decentralized and

*Email addresses:* m.s.baumgartner@student.vu.nl (Meret Baumgartner), m.h2.karsten@student.vu.nl (Max Karsten), i.malavolta@vu.nl (Ivano Malavolta)

commercial nature of the cloud [3]. According to various surveys, more than 70% of IT organisations use microservices [4, 5, 6]. This suggests that not just cloud computing in general, but microservice applications in particular, form relevant objects of study with regards to their energy usage [7]. While microservices have been widely adopted because the loosely coupled, small and lightweight components are scalable, flexible and can be used to increase resilience, their deployment also has caveats, such as increased overhead and complexity [8]. In order to understand and reduce the resource utilization and environmental impact of microservices, realistic simulation of such architectures in cloud datacenters is important [9]. Through simulation, potential areas of improvement to reduce energy usage can be identified and strategies to reduce resource consumption can be tried out at lower cost [10]. A large number of cloud simulators is available, but they all differ in the capabilities they offer, components they model, their implementation and ease of use.

In this study, we compare five cloud simulators, *CloudSim*,*DISSECT-CF*, *NetworkCloudSim*, *OpenDC* and *SimGrid* that offer the capability to simulate energy usage[1]. These simulators were selected by extracting 67 simulators from 23 peer-reviewed articles reviewing cloud simulators and choosing the five most recently updated simulators that have energy usage simulation capabilities and for which the code is publicly accessible. Notably, none of the considered simulators provides support for modeling energy consumption at the level of individual microservices. As a result, the scope of this study is limited to the estimation and evaluation of total host-level energy consumption. Then, we assess the simulators according to the accuracy of the simulated host-level energy values they produce for traces of two different microservice applications, *Sock Shop*[2] and *Train Ticket* [11] by comparing the results to real-world measures. Furthermore, we describe the extent to which the simulators support energy-related simulation through what resources can be simulated but also what other capabilities related to energy are offered. While the software systems the simulators are set to simulate are microservices, we also look at the affordances of the simulators themselves and how well they are suited to simulating microservice applications.

Our results show that *OpenDC* is the simulator most suited to simulating host-level microservice applications, both in terms of accuracy of its energy usage simulations and its suitability to simulating microservice architecture style applications. It has both the lowest minimum (0.14) and maximum (0.41) mean relative error and is the second-fastest simulator with simulation times of less than one second. *NetworkCloudSim* has both the highest minimum (0.42) and maximum (0.95) mean relative error. *SimGrid* without simulating communication is the fastest simulator with a mean simulation time of 0.04 seconds, while it is the slowest (12.02 seconds on average) when including communication in its simulation. Depending on the context such as workload definition, user skill and preference, and desired levels of configurability, other simulators might be well suited as well. By doing so, we enable prospective users of cloud simulators the possibility to make an informed choice about what cloud simulator to use, and perhaps even make it more attractive to start to do so. Furthermore, making a more informed choice in cloud simulator could lead to more accurate simulation results as we show that the simulator currently most used, also in the context of microservices [7], *CloudSim*, is not always the best choice to make. Researchers and engineers working with microservice systems can benefit from this study as it can help them with making the choice of cloud simulator easier and more informed, in turn enabling them to design and develop more energy-conscious microservice systems.

## 2. Background

### *2.1. Microservice Architecture Style*

In the microservice architecture style, applications are broken up into loosely-coupled independent components that communicate with each other and split the workload according to capability. Computation is focused in the services with communication between services being light-weight, often through APIs over HTTP. Due to this decentralized modular nature microservice systems are more scalable, robust and more

[1]The replication package for this study can be found at https://github.com/S2-group/simpat-2026-cloud-simulators-energy.
[2]https://github.com/microservices-demo/microservices-demo

easily modifiable [12]. Since services are independent, bugs or faults are less likely to propagate. Furthermore, services can be independently scaled up and down and deployed, making it more flexible [13]. The microservice architecture style does not necessarily decrease complexity, however. The strategies employed for managing services and the resources they run on can even lead to increased complexity and cost [8].

### *2.2. Energy-efficient Microservices*

Studying and reducing the resource consumption, and particular the energy usage of the IT industry is a complex but crucial task [14]. The flexible and distributed nature of microservices form both opportunities and challenges in this regard. Scalability and flexibility mean that resources can be allocated resources according to demand, but this also requires in increase in complexity and overhead. Furthermore, techniques that may decrease energy usage, may reduce performance, reliability and availability. This is why new innovative techniques specifically designed for energy usage reduction in microservices are needed [7, 15].

#### *2.2.1. Measuring Energy Consumption of Microservice-based Systems*

In general, energy consumption measurement methods can be classified into three categories: measurement through external hardware, modelling power consumption based on resource utilization, and software solutions utilizing available hardware sensors to report energy consumption [16]. In the microservices context, measurement of energy consumption can be divided into system-level and service-level measurement. As external hardware such as power meters only provide measures on a host- or system-level, they are not well-suited to microservice-based systems [17]. The method of modelling power consumption is employed (on a very basic level) by many simulators such as those discussed in this study, while software solutions are used to collect the ground truth these modelled values are compared to here. Examples of software energy measurement tools are Perf[3], Scaphandre[4], Likwid[5] and PowerJoular[6] which all use the Running Average Power Limit (RAPL) sensors which can report power consumption on the level of individual CPU sockets up to the entire system[18]. Next to this, other software tools for measuring energy consumption of, for example, Python applications (e.g., EnergyScope[7], PyJoules[8]) or GPUs (e.g., nividia-smi[9]) exists.

### *2.3. Cloud Simulators*

Cloud simulators are used to simulate the behaviour of cloud datacenters and software or tasks running on them. Simulation can help with designing systems or applications, testing strategies and policies, and finding areas of improvement in current implementations. Pertinent topics of study at the moment are performance on large scales, security and resource usage in order to bring both environmental and monetary cost down [10]. There is a substantial group of cloud simulators that support some form of energy usage simulation, but as these often originate out of scientific research, they are not always maintained after initial publication or cease being publicly available at all. Furthermore, not all cloud simulators are well suited to simulating microservices due to the containerized nature of the architecture style. Finally, the configurability of simulators can vary drastically. This can range from how fine-grained the user must define the system topology to how workloads can be defined in the simulator. *CloudSim* [19] is the most popular cloud simulator [20], a Java-based simulator that is relatively simple itself but for which extensions for *e.g.,* communication and GPU simulation are available and that other simulators such as *CloudSim Plus* [21] are based on. *GreenCloud* [22] is a simulator first published in 2012 and one of the earliest simulators to include energy simulation capabilities for servers, but also network links and switches. It is not maintained any more. Table 12 gives an overview of 67 simulators considered during the simulator selection phase of this experiment, which is described in subsection 5.2 along with a description of *CloudSim*, *DISSECT-CF*, *NetworkCloudSim*, *OpenDC* and *SimGrid*, the simulators that were finally selected for this study.

[3] https://perfwiki.github.io/main
[4] https://github.com/hubblo-org/scaphandre
[5] https://github.com/RRZE-HPC/likwid
[6] https://github.com/joular/powerjoular
[7] https://www.energyscope.net/latest
[8] https://github.com/powerapi-ng/pyJoules
[9] https://docs.nvidia.com/deploy/nvidia-smi/index.html

## 3. Related Work

Araújo et al. [7] give an overview of research concerning energy consumption by microservice applications through an extensive literature review. While it was the most evaluation method used by most papers, the authors note that 10% of papers that simulate microservices do not specify what simulator they use or how the microservices are simulated. In the papers that do disclose the simulators used, *CloudSim* is by far the most frequently used, followed by the *DRACeo* simulator, the *Linpack* benchmarker and finally *Matlab*. This study shows that simulators, particularly *CloudSim* are utilized by researchers to investigate energy consumption of microservice applications. Furthermore, Araújo *et al.* note that authors seem to find the simulators used inconsequential to their research. The research discussed here builds on these observations by investigating what cloud simulators are well-suited to simulate microservice applications and showing how accurate they are. This also highlights that what simulator is chosen can indeed have an impact on the results obtained through its use.

Mansouri et al. [10] present an extensive review of 32 cloud simulators including *CloudSim*, *NetworkCloudSim* and *DISSECT-CF* describing their implementations, strenghts, limitations, and available features, including whether they provide energy modelling. They note the importance of including energy consumption simulation in cloud simulators and that the simulators that do provide this only do so in a quite simplified manner, often only taking CPU utilization into account. While they do consider metrics of the simulators themselves such as their time and energy consumption, they do not consider the accuracy of the simulated results. They do note that the simulators are not very scalable, which is an important capability with cloud systems rapidly increasing in scale. The research presented here builds on this paper by focusing on the energy consumption simulation capabilities of a smaller selection of cloud simulators.

Papers by Ismael [23] and Vincenzo Emanuele Martone et al. [24] both describe cloud simulators with a focus on their abilities to simulate energy consumption metrics. They are both literature reviews that extract their information from other reviews, simulator papers or general properties of the simulators and do not test the simulators themselves as opposed to the simulated metrics as is done in the research discussed here. In [23], eleven simulators including *CloudSim* and *DISSECT-CF* are discussed. Simulators are evaluated in terms of the hardware components for which energy consumption can be simulated, what energy metrics can be extracted, and energy management techniques that are provided to lower the energy consumption. Furthermore, the impact or popularity of the simulators is discussed, specifically whether the simulators were used or extended in subsequent works. Finally, Ismael [23] notes that most simulators only support energy simulation of certain components, not of complete systems through the entire energy chain. Furthermore, there is a lack of support for multi-objective simulation and optimization, as simulators that support energy simulation often do not support e.g. modelling of network communication.

Martone et al. [24] describe five simulators, again including *CloudSim* and *DISSECT-CF* according to their implementation and energy simulating capabilities and evaluates them according to when they were last updated, popularity, availability, granularity and performance profile. For the last metric, values such as simulators' run times and memory usage are taken from other papers. Finally, they recommend *CloudSim* as the generally most suitable simulator for general-purpose simulation of cloud systems given its widespread adoption, extensive documentation and numerous available available extensions.

In a recent paper, Hasanein Rjeib and Gabor Kecskemeti [25] do perform evaluations of the accuracy of simulated energy consumption measures by *CloudSim* and *DISSECT-CF* using the PlanetLab workload traces. For these traces, they find that *DISSECT-CF* simulates energy consumption more accurately, in part because it also provides the capabilities to simulate the energy usage of resources such as RAM. Furthermore, they observe some abnormal behaviour by CloudSim when CPU usage is 0 for a VM. They note that, while CloudSim is simpler to implement and easier to use, DISSECT-CF has a higher accuracy and lower simulation time, making it overall the preferred simulator. This paper is somewhat similar to the research described here but differs as it it only considers two simulators and does not test microservice applications. Moreover, it only very briefly discusses the usability of the simulators.

While previous work such as that discussed in this section has compared cloud simulators on a higher level, both for general use cases and focused on simulating energy usage, this study goes further by actually carrying out simulations and providing the produced energy values to real-world measures. Furthermore,

this is done in the context of microservice applications, which is a context in which cloud simulators are used to support research, but are very rarely the subject of study themselves.

## 4. Goal and Research Questions

The aim of this study is to evaluate the simulated energy consumption of microservice applications reported by cloud simulators by comparing it to real-world measures. In order to further refine this goal, the GQM approach [26] (see Table 1) is used and multiple research questions are formulated.

Table 1: GQM Research goal definition.

| | |
|---|---|
| **Analyze** | Open-source Cloud simulators |
| **for the purpose of** | evaluating |
| **with respect to their** | suitability of simulating energy usage |
| **from the point of** | software developers, researchers |
| **in the context of** | microservice applications |

**RQ1**: *What is the accuracy of the selected cloud simulators for simulating host-level energy consumption?* This research question focuses on the energy consumption simulation capabilities of the cloud simulators. It can help engineers and researchers select a simulator and give an indication of how to interpret energy consumption simulated by the selected simulators. This research question can be divided into the following two sub-questions:

**RQ1.1**: *How accurate are the selected cloud simulators in terms of measuring the host-level energy consumption of microservices?*

**RQ1.2**: *How does the accuracy of the selected cloud simulators vary when using different power models?*

**RQ2**: *What are the main characteristics of the selected cloud simulators and how suitable are they for simulating microservice applications?* This research question focuses on the affordances of the cloud simulators for simulating microservices. This is particularly relevant as the selected simulators were not designed explicitly with microservice applications in mind. Answering this question can help engineers and researchers select a simulator given the systems they want to simulate. This research question can be divided into the following two sub-questions:

**RQ2.1**: *What degrees of customizability do the simulators provide?*

**RQ2.2**: *What is the ease of use of the simulators?*

## 5. Experiment Plan

### *5.1. Applications Under Test*

The two microservice applications used to test the accuracy of the simulators are *Sock Shop* [27] (see Figure 1) and *Train Ticket* [28] (see Figure 2). An overview of properties that can be used to describe these applications can be found in Table 2. Both simulate e-commerce application for online stores consisting of services that communicate with each other via REST API over HTTP. While we can see from the number of services and different programming languages used that *Train Ticket* is more complex. *Sock Shop*, on the other hand, is far more popular and regularly used in research, including recent research into topics related to the energy consumption of microservices [29, 30].

[10] Retrieved 20 June 2025.

Table 2: Description of the microservices applications used for testing.[10]

| **Property** | ***Sock Shop*** | ***Train Ticket*** |
|---|---|---|
| Created | 2016 | 2019 |
| Last updated | 2023 | 2022 |
| Number of services | 13 | 59 |
| Programming languages | Java, JavaScript, Go | Java, JavaScript, Python, Go |
| Types of databases | Mongo, MySQL | Mongo, MySQL |
| Communication protocol | HTTP | HTTP |
| Commits | 1705 | 323 |
| Contributors | 55 | 26 |
| Forks | 2875 | 272 |
| Stars | 3702 | 788 |

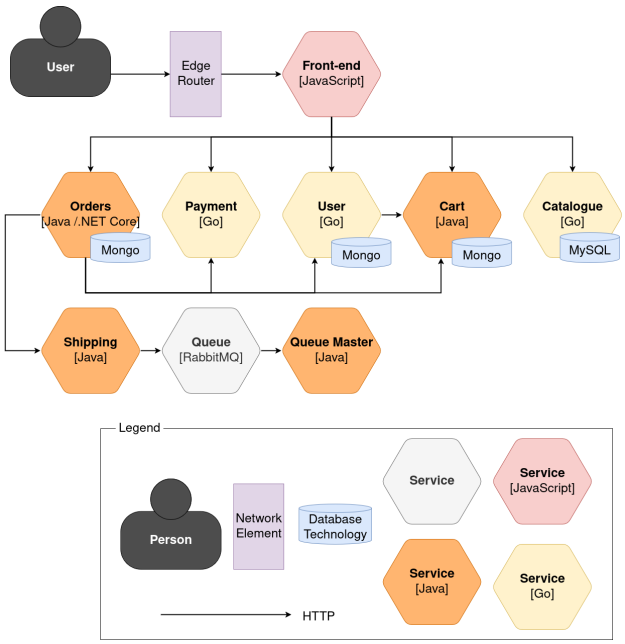



Figure 1: System architecture of Sock Shop.

### *5.2. Simulators Under Test*

#### *5.2.1. Simulator Selection*

In order to select the simulators analysed in this study, a systematic selection is carried out (see Figure 3). First, 23 papers are collected by querying Google Scholar[11] for *Cloud computing simulators review* with search period 2019-present, *Cloud computing simulator*, and *"cloud computing simulators"* with search period 2020-present. The results are ordered by relevance and the top results are collected[12]. Furthermore, the forward snowballing method [31] is used by collecting papers that cite [10][13], as this is the most influential relevant paper obtained through the previous queries having been cited more than 100 times. This results in a total of 23 papers, 13 of which are determined to be relevant as discussing multiple cloud simulators and their properties (see Table 11). From the 13 papers, all simulators mentioned are extracted as well as 10 simulators that are suggested by a research practitioner in the field of microservices. This leads to a total

[11] https://scholar.google.com
[12] This was done on 4 February 2025.
[13] https://scholar.google.com/scholar?cites=8172644899227091971

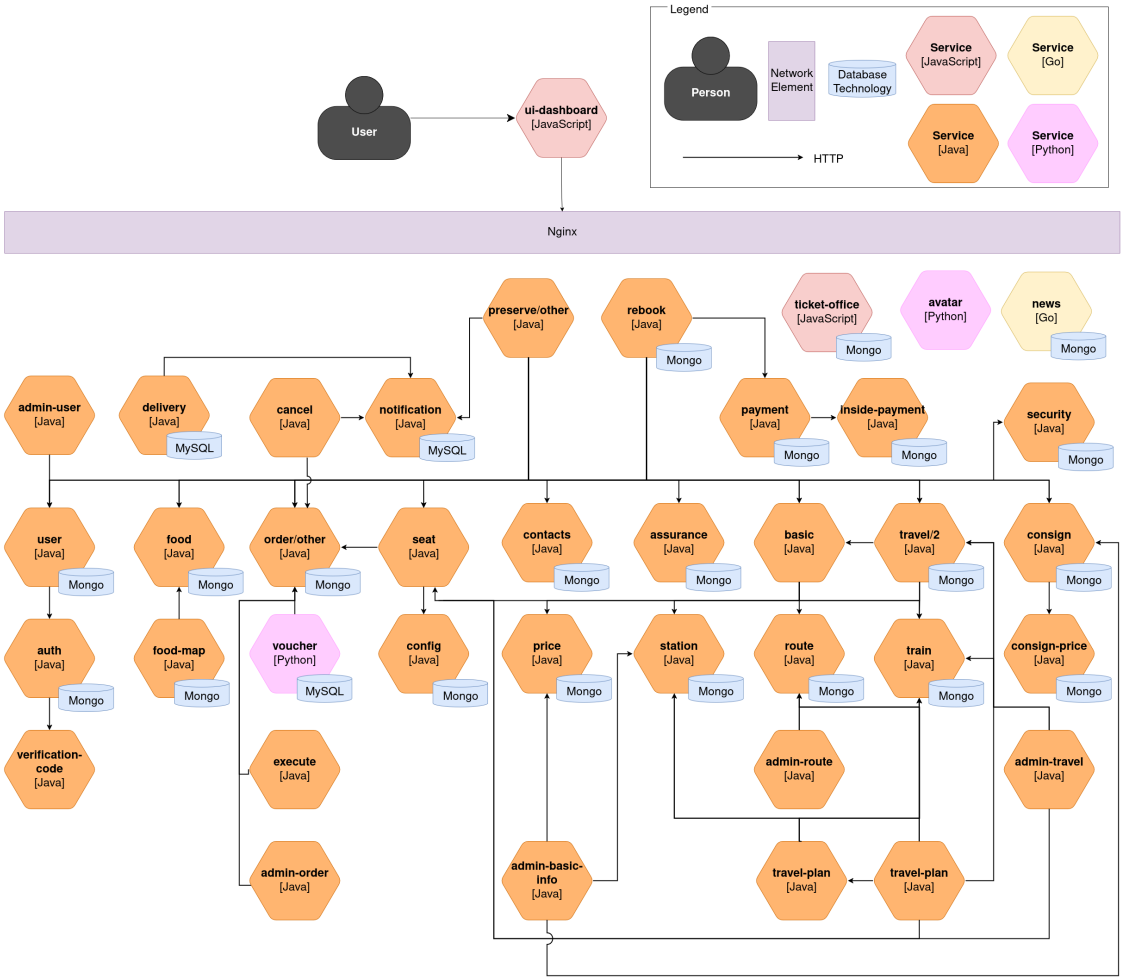



Figure 2: System architecture of Train Ticket.

of 67 simulators for which the date of the first publication presenting the simulator and the date of the most recent update to the simulator are retrieved (see Table 12).

Furthermore, each simulator is assessed based on whether its code is publicly available and whether the simulator has energy usage simulation capabilities. Only simulators for which the code is publicly available are considered both to increase the replicability of this study as well as to be able to inspect their code. 37 simulators fulfil both these requirements. From the 37 simulators that fulfil both these requirements, the 6 simulators that have been most recently updated are selected: *CloudSim*, *NetworkCloudSim*, *OpenDC*, *SimGrid*, and *DISSECT-CF*. Table 3 gives an overview of their properties. While *CloudSim Plus* was also recently updated, it was not included in this study as it runs on the *CloudSim* engine, which is already included in the study through both *CloudSim* and *NetworkCloudSim*. Out of the five selected simulators, *CloudSim* is by far the most popular simulator with the largest number of forks, stars and citations, while *OpenDC* is the newest simulator, having first been published in 2021. In remainder of this section, a general description of each simulator is given.

### *5.2.2. CloudSim*

*CloudSim* [36] is a simulation toolkit for simulating cloud environments, built on the *SimJava* [37] discrete event simulation engine and the *GridSim* [38] Grid simulation toolkit. It was first published in 2009 and is still being developed by the CLOUDS Laboratory at the University of Melbourne. It is written in Java and

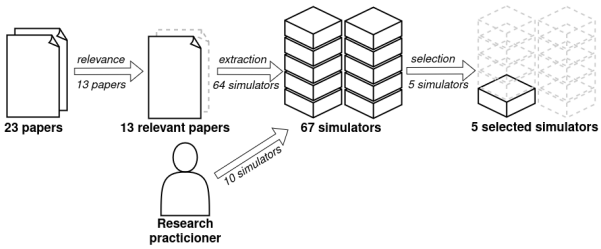



Figure 3: Simulator selection process.

Table 3: The selected cloud simulators[2].

| **Property** | **CloudSim** | **DISSECT-CF** | **OpenDC** | **NetworkCloudSim** | **SimGrid** |
|---|---|---|---|---|---|
| Created | 2009 | 2015 | 2021 | 2011 | 2001 |
| Last updated | 2025 | 2023 | 2025 | 2025 | 2025 |
| Based on | *SimJava*, *GridSim* | - | - | *CloudSim* | - |
| Commits | 465 [32] | 466 [33] | 2267 [34] | 465 [32] | 37573 [35] |
| Contributors | 15 | 4 | 17 | 15 | 70 |
| Forks | 532 | 44 | 62 | 532 | 179 |
| Stars | 912 | 6 | 86 | 912 | 94 |
| Programming languages simulator | Java | Java | Kotlin | Java | C++, C, Python, or Java |
| Programming languages simulation | Java | Java | - | Java | C++, C, Python, or Java |
| Citations first paper[3] | 1724 | 125 | 37 | 446 | 695 |

implements `Brokers` and `Datacenters`, with the latter being assigned stand-alone objects such as `Hosts`, `VMs` and processing units called `Cloudlets`. For the allocation and scheduling of tasks, provided allocation and scheduling policies can be used, or they can be custom defined by the user. Workloads can be loaded from files in *Standard Workload Format* [39] (swf) or defined by the user in code.

The capability to simulate energy usage within CloudSim was added in 2012 [40], with dedicated power objects such as
`PowerDatacenter` and `PowerHost`. For the calculation of the simulated energy usage, only the hosts' CPU usage is taken into account. Four power models (cubic, linear, square root and square) are provided, and the user can define a custom power model.

### *5.2.3. DISSECT-CF*

*DISSECT-CF* [41] is an IaaS simulation framework, first published in 2015. It was developed with a focus on modelling energy usage and evaluating different scheduling strategies. It is written in Java and contains five subsystems of decreasing level of abstraction: an event system, unified resource sharing, energy modelling, infrastructure simulation and infrastructure management. Workloads can be loaded from files in gwf, swf, srtg or one2 format, or defined in code. For the simulation of energy usage, *DISSECT-CF* decouples energy consumption from resource utilization. Instead, it uses resource utilisation counters that are updated depending on the power state of a resource. For each individual power state, a different power consumption model is defined. Predefined power states for a physical machine are *off*, *switching on*, *running* and *switching off*. *DISSECT-CF* provides three predefined power models, constant, linear and NoIdle, a variation on the linear model, as well as the possibility for the user to define their own custom power model.

### *5.2.4. NetworkCloudSim*

*NetworkCloudSim* [42] is an extension to the *CloudSim* framework that enables the simulation of network components and message passing, first published in 2011. It is also implemented in Java with dedicated network objects, such as `NetworkDatacenter` and `NetworkHost`. In their original implementation, these objects are incompatible with the dedicated power objects, but their capabilities do not interfere with each other.

Because of this, we combined the power and network elements where necessary and created for example a `PowerNetworkHost` in order to still be able to evaluate *NetworkCloudSim*. As we only combined existing objects and did not add any energy usage simulation capabilities, this means that also for *NetworkCloudSim* only CPU usage is taken into account and the same four power models are available as for *CloudSim*, next to the possibility for the user to define a custom power models. As is the case for *CloudSim*, workloads can be given in swf format or defined by the user. Communication is defined as actions between `Cloudlets` and have to be defined in code.

#### *5.2.5. OpenDC*

*OpenDC* [34] is a platform for datacenter simulation, first published in 2021. It is implemented in Kotlin, but experiments on the platform and their topologies are supplied as JSON files. This leads to the platform having a high level of abstraction and a low level of customizability. Simulated workloads are given as parquet[4] files. To calculate the simulated power usage, only CPU usage is taken into account and *OpenDC* provides five power models: constant, which is the default, cubic, linear, square root and square. It is not possible for users to define their own power models. OpenDC does provide the option to include carbon traces, which are then used to calculate the carbon intensity alongside the energy usage.

#### *5.2.6. SimGrid*

*SimGrid* [43] is a parallel and distributed computing (PDC) simulator, first published in 2001[44]. It is available in C++, C, Python and Java, we used the version written in C++ for this study as it is the most extensive. Workloads have to be defined in code. The energy usage simulation capabilities of SimGrid are defined in the *Host Energy* plugin [45] and *Link Energy* plugin. Next to these plugins, there are also *WiFi Energy*, *Battery*, *Solar Panel* and *Chiller* plugins, but these will not be studied here. Energy consumption is modelled linearly, based on off, idle, maximum utilization, and an `Epsilon` value for a near-idle state. Energy consumption for a certain CPU usage is calculated through linear extrapolation between the `Epsilon` and maximum values.

### *5.3. Experiment Variables*

### *5.4. Independent Variables*

For both RQs, the subjects are the cloud simulators (see subsection 5.2) tested. For these simulators, various different simulation configurations are tested (see Table 4).

For *CloudSim* and *NetworkCloudSim*, workloads are defined in code with one `Cloudlet` per service. For *SimGrid*, an actor is created for each service which is then set to execute computations according to the CPU utilization from the workload's trace file. For each simulator, all available predefined power models are tested. Custom power models are not implemented and then tested as these depend too much on the users' implementation and are therefore not representative of the simulator, which is the main object of study.

The microservice applications (see subsection 5.1) and their treatments are the other independent variables. The treatments are based on previous work that involved measuring the energy consumption of *Sock Shop* and *Train Ticket* [46]. They consist of two scenarios and two different user loads. The user loads are 100 and 1000, which are achieved by gradually increasing the number of users at a rate of 5 users per second.

#### *5.4.1. Dependent Variables*

The aim of this study is to evaluate the selected simulators for their use to simulate the host-level energy consumption of microservice applications. For RQ1, the accuracy of the simulators and their predefined power models in terms of simulating host-level energy consumption is measured by calculating the mean relative error (MRE) over the $N$ executions of each simulator with respect to the measured energy consumption (see Equation 1). The main dependent variable for this study is therefore the simulated host-level energy consumption in Joules.

[2] See footnote 10.
[3] Obtained using the Google Scholar "cited by" function.
[4] https://parquet.apache.org/

Table 4: Simulation configuration variables tested for the simulators.

| **Configuration variable** | ***CloudSim*** | ***DISSECT-CF*** | ***NetworkCloudSim*** | ***OpenDC*** | ***SimGrid*** |
|---|---|---|---|---|---|
| Workload definition | code | swf trace | code with communication | parquet trace | code<br>code with communication |
| Power models | linear<br>square root<br>square<br>cubic | constant<br>linear | linear<br>square root<br>square<br>cubic | constant<br>linear<br>square root<br>square<br>cubic | linear |
| Total configurations | 4 | 2 | 4 | 5 | 1 |

$$MRE = \frac{1}{N}\sum_{i}^{N}\frac{|\text{measured energy}_i - \text{simulated energy}_i|}{\text{measured energy}_i} \tag{1}$$

For RQ2.1 concerning the customizability of the simulators, the elicited information are the properties that *can* and *have to* be set, for both the hardware system and workload to be simulated, as well as through what method. For RQ2.2 regarding the usability of the simulators, the mean runtime of the simulators, as well as the quality of the simulator's documentation are taken into account. This quality is assessed based on the availability of documentation, whether it is up to date, and whether it covers the energy consumption simulation features of the simulators.

### *5.5. Experiment Design*

To obtain the ground truth against which the simulators are compared, the systems under test are measured with the treatments described in subsection 5.4 using a cross-over design [47]. All possible combinations of 2 applications × 2 scenarios × 2 user loads result in 8 different treatment combinations that are executed in a randomized fashion. This is to ensure that the order of execution does not introduce bias while possible influences from the environment, network or system are not localized to one treatment. Furthermore, each treatment is executed 30 times with a 3-minute warm-up period before 5 minutes of measuring and finally a 2-minute cool down period.

## 6. Experiment Execution

### *6.1. Subjects Preparation*

In order to not just look at workloads from two different microservice systems, but also at different levels of complexity and behaviour patterns within these microservice systems, two usage scenarios are defined for each system. The two usage scenarios for *Sock Shop* and *Train Ticket* are comparable across systems. The first scenario for *Sock Shop* involves opening the catalogue and randomly selecting an item for preview. In the second scenario, the user then logs in, opens the catalogue and adds a random number of items to cart to create an order with. For *Train Ticket*, a user adds users to the database, logs in and then searches for tickets in the first scenario. In the second scenario, the user again adds users to the database and logs in, but this time they search for a train ticket and book it. Across both scenarios and applications, users randomly wait 1-3 seconds between carrying out any action. Furthermore, the random numbers used throughout these scenarios are always picked from a range of 3 to 9. To script these scenarios, Locust.io[5] is used.

[5] https://locust.io

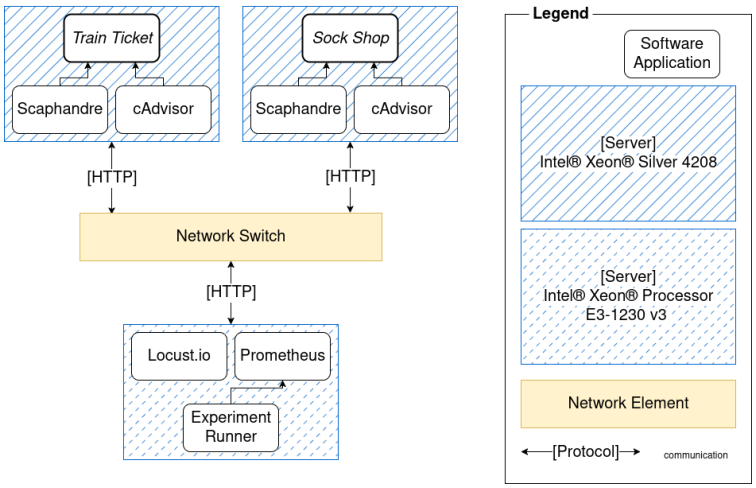



Figure 4: Setup of the servers used to collect ground truth measures.

### *6.2. Setup*

#### *6.2.1. Collecting Ground Truth Measures*

To collect the ground truth measures, *Sock Shop* and *Train Ticket* are deployed on two servers provided by the *Green Lab*[6]. These servers each run Ubuntu 22.04.2 LTS on Intel Xeon Silver 4208 CPU @2.10 GHz with 16 cores and 384 GB of RAM. On these servers, Scaphandre[7] is used to collect energy consumption measurement values in $\mu W$ of the services individually and the system overall every minute. Scaphandre does so by combining the per-socket RAPL with the DRAM RAPL power measures. To collect metrics regarding resource usage such as CPU and memory usage, cAdvisor[8] is used. In order to make sure it does not interfere with the measurement, the collection of those measures is done on a third server running on an Intel Xeon Processor E3-1230 v3 @ 3.40 GHz with 8 vCPUs, 32 GB of RAM and 1 TB of HD, also operating on Ubuntu 22.04 LTS. These three servers are connected to a switch (see Figure 4) and share a bandwidth of approximately 1 Gbps and a latency of approximately 0.25 ms. For the collection of measures from the other two servers, Prometheus[9] is run on this server. The systems on the servers are deployed using Docker version 27.3.1.

#### *6.2.2. Simulation Executions*

Using Python version 3.10.4, swf and parquet traces are produced from the obtained resource mesures for each treatment consisting of one of the microservice applications, scenarios, user loads and repetitions. From the Locust logs, some high-level communication traces are produced. Given that distributed tracing is not supported for the microservice applications, only requests to and from the applications are logged.

Execution of the simulations is done on an additional workstation server containing Intel Xeon Silver 4208 CPU @ 2.10 GHz with 16 cores and 384 GB of RAM. This was to run each simulator in a best case scenario where they were not limited by resources. The simulators were run on Ubuntu 22.04.2 LTS with Python 3.10.4, Java (OpenJDK) 21.0.7, and g++ 11.4.0.

### *6.3. Execution Phase 1: Ground Truth Collection*

For collecting the ground truth, every trial is run 30 times in total. Each experimental run begins with a three minute warm-up period to make sure stable idle cpu utilization levels. A treatment is then randomly

[6] https://s2group.cs.vu.nl/pages/greenlab
[7] https://github.com/hubblo-org/scaphandre
[8] https://github.com/google/cadvisor
[9] https://prometheus.io

selected, ensuring that the order of trials does not bias the results. The trial's user scenario is then run via locust for 5 minutes, after which data is collected with Prometheus. Finally, a cooldown period of 2 minutes is initiated to ensure the system returns to idle levels. The warm-up and cooldown periods ensure that trials do not influence each other and are as accurate and reliable as possible. Prometheus is used to collect power consumption metrics from Scaphandre at 5-second intervals as well as service-level metrics from cAdvisor such as CPU utilization and memory usage. These values are queried from Prometheus using PromQL[10].

#### *6.3.1. Trace Construction*

The values queried from Prometheus are saved and later used to generate traces. In order to reduce the large number of traces ($2 \cdot 2 \cdot 2 \cdot 30 = 240$), for each treatment, the four repetitions with the total energy consumption closest to the median are selected. The resulting selection of traces is then simulated with the configurations described in Table 4, resulting in $32 \cdot (4 + 2 + 4 + 5 + 1) = 512$ simulation instances that are each repeated 30 times in order to account for any possible non-deterministic behaviour from the simulators. In particular, CPU and memory utilization are used to generate `swf` and `parquet` traces, as well as defining the described workloads in code. For the `swf` traces, each 5-second measurement of a service is defined as a job in order to preserve the degree of granularity of the CPU and memory utilization measures.

For the communication traces, for each trace, the mean response size is calculated and saved. The communication trace itself consists of the number of requests per second time step.

#### *6.3.2. Ground Truth Construction*

The power consumption samples collected from Scaphandre using Prometheus is used to build the ground truth dataset against which the simulated energy consumption values are compared. As the simulators produce energy consumption values on a host-level, we are using the power samples that describe the energy usage of the whole system. In order to do so, the obtained power consumption values are transformed from $\mu W$ to Joules consumed in that time step as described in Equation 2. The power consumption values are transformed from $\mu W$ to $W$. Then, the trapezoid rule is applied by computing the mean of the power values at the beginning ($p_{i-1}$) and end ($p_i$) of each time step, and multiplying this by the duration of the time step in seconds to estimate the energy consumed during that interval and adding this value to the calculated energy consumed during the previous intervals. Each service is given a number and jobs are allocated to the service by giving this number both as its group ID and executable number.

$$\text{Energy}(t_0, t_N) = \int_{t_0}^{t_n} p(t)\, dt \approx 10^{-6} \sum_{i=0}^{n} \frac{p_{i-1} + p_i}{2} (t_i - t_{i-1}) \tag{2}$$

Table 5: Mean relative error for the selected simulators per power model. Bold values are a simulator's lowest MRE.

| | ***CloudSim*** | ***NetworkCloudSim*** | ***OpenDC*** | ***DISSECT-CF*** | ***SimGrid*** | | **Mean** |
|---|---|---|---|---|---|---|---|
| | code | code with communication | parquet | swf | code | code with communication | |
| linear | 0.74 | 0.74 | 0.41 | 0.71 | **0.42** | **0.42** | 0.57 |
| square | 0.63 | 0.63 | 0.41 | - | - | - | 0.56 |
| cubic | **0.54** | **0.54** | 0.41 | - | - | - | 0.50 |
| square root | 0.80 | 0.81 | 0.37 | - | - | - | 0.66 |
| constant | - | - | **0.14** | **0.44** | - | - | 0.29 |
| Mean | 0.68 | 0.68 | 0.35 | 0.57 | 0.42 | 0.42 | |

### *6.4. Execution Phase 2: Simulations Executions*

In order to execute the simulations on the selected simulators, the treatments described in Table 4 are each run 30 times to increase reliability and deal with any intrinsic non-determinism of the simulators. The simulated energy usage for each run is written to file along with the runtime of each simulation.

[10] https://prometheus.io/docs/prometheus/latest/querying/basics

#### 6.4.1. System Properties Collection

In order to be able to define the systems used to collect measures for simulation, some properties have to be collected. As a measure of CPU performance, the Instruction Per Cycle (IPC) or variations of it are needed for all simulators except *OpenDC*. This metric is not just dependent on the system but also the applications running. In order to get an approximation of this value, the IPC of the servers running the applications and monitoring tools was calculated using the Linux `perf stat` command over 1 minute and repeated 30 times for both *Sock Shop* and *Train Ticket*. The mean of these 30 IPC values was then taken to be used to configure the cloud simulators. For configuration of the simulators' power models, power consumption measures at three different CPU utilization levels have to be collected: *off*, *idle* and *max*. The off level refers to the powerdraw when it is completely off. As we are collecting the energy usage from the server itself and not its external powerdraw for this study, this value is set to 0. For the idle level, we take the mean of host power values for the time steps of the ground truth dataset for which the sum of CPU usage over all monitored containers is 0%. For the max level, we do the same but for a CPU usage of 100% or more, which occurs without additional interference.

#### 6.4.2. CloudSim

Workloads are defined in code for *CloudSim*. In order to do so, a `PowerContainer`, `PowerContainerVM` and `Cloudlet` are created for each service and a custom `UtilizationModel` is defined to provide the CPU utilization per time step for the given service to the `Cloudlet` from the trace. While *CloudSim* does provide a way to load workloads as `swf` files, it only takes the id, runtime and number of processors fields into account, meaning that e.g. task submission in time is not possible. Because of this, we decided not to simulate workloads as swf files for *CloudSim*.

#### 6.4.3. NetworkCloudSim

The manner in which workloads defined in code for *NetworkCloudSim* is similar to that for *CloudSim*. For each service, a `PowerContainer` and `PowerContainerVM` are created, as well as a `NetworkCloudlet`. The custom `UtilizationModel` defined for *CloudSim* is reused. For the communication, two extra `NetworkCloudlets` are created representing the requests and responses. For each request from the trace, a request and response with the mean response size are added to these two `NetworkCloudlets`. In order to be able to run the `NetworkCloudlets` in combination with energy modelling, the custom `PowerNetworkHost`, `PowerSwitch` and `PowerContainerNetworkDatacenter` classes are defined. These classes only make the combination of the power and network modules and do not add extra capabilities or logic.

#### 6.4.4. DISSECT-CF

Workloads are given to *DISSECT-CF* as `swf` trace. To load the `swf`, the provided `FileBasedTraceProducerFactory` is used. As the energy measures that the simulated values will be compared to are only from the CPU, the powerstates of the other resources such as storage and network are set to 0 and only the consumed energy of the hosts are registered.

#### 6.4.5. OpenDC

Workloads are given to *OpenDC* as two `parquet` files. The services are defined in the *tasks* file, while the CPU usage metrics of the services over time are given in the *fragments* file. With *OpenDC*, it is possible to execute multiple workload simulators in batch during one experiment, in this case of 8 workloads. In order to obtain the simulation time of each workload, the simulation time of the experiment is divided by the number of workloads simulated.

#### 6.4.6. SimGrid

For SimGrid, the workloads are given in code, both with and without communication. This is done using the S4U interface and the *Host Energy* plugin. For each service, an S4U agent is defined with a function that reads the CPU consumption values for that service. From the CPU consumption value, the actual runtime of the CPU for that time step is calculated and the service is set to sleep for the remainder of the time step. To

Table 6: Mean standard deviation of simulated power consumption in Watt over runs of the same input workload.

| Simulator | Mean std |
|---|---|
| *CloudSim* | 0.089 |
| *DISSECT-CF* | 0.484 |
| NetworkCloudSim | 0.089 |
| *OpenDC* | 0.160 |
| *SimGrid* | 3.412e-6 |

add communication, an extra agent is added on the original host which responds to requests with the mean response size. To send these requests an additional host was created with an agent to generate them. The communication between these agents was modelled using the synchronous communication interface provided by SimGrid. Only the original host's energy consumption is recorded.

## 7. Results

### *7.1. RQ1: Suitability for Simulating Energy Usage*

#### *7.1.1. RQ1.1 Accuracy of Simulators*

In Table 5, we can see an overview of the mean relative error per power model for the selected simulators. There, we can see that *OpenDC* consistently is most accurate across all tested power models. It is followed by *CloudSim* for the cubic power model and then *NetworkCloudSim* for the cubic and *SimGrid* for the linear power model. In the table, we can see that the MRE for *SimGrid* is the same for both workload definitions both with and without communication. Indeed, using the Mann-Whitney U test, we obtain a p-value of 1.0, which indicates that there is no evidence of a significant difference between the two workload definitions. By performing the test on the simulated values of *CloudSim* and *NetworkCloudSim*, we obtain a p-value of 0.31 indicating that there is not a significant difference between the results of these simulators. However, because in previous test runs the p-value reported as 0.0 the *NetworkCloudSim* results were included for comparison. By performing the Kruskal–Wallis test on all five simulators, we obtain a p-value of again 0.0, indicating evidence of a significant different between the simulators.

Between *CloudSim* and *NetworkCloudSim*, the MRE seems to be very similar as well. As we can see in Figure 6, *NetworkCloudSim* consistently produces slightly higher energy usage values than *CloudSim*. It is also notable that in the same figure, we can see that both *CloudSim* and *NetworkCloudSim* overestimate the energy usage. For the linear power model, *OpenDC* overestimates the energy usage, while it underestimates it for the other power models. *DISSECT-CF* on the other hand underestimates in the constant model but overestimates in the linear model. Furthermore, we can see that at until about half-way through the measuring window, the *OpenDC* with the non-constant models, *SimGrid* and even *DISSECT-CF* with the constant power model have a smaller error. As the total energy starts to increase more rapidly, *OpenDC* with the constant power model becomes the most accurate.

As discussed in subsubsection 7.1.1, the power model that most accurately simulates the power usage seems to change over time. This suggests that dynamic power models could estimate energy usage more accurately over time. It is notable that, even while the simulator-power model combination that simulates the energy usage most accurately changes over time, *OpenDC* is consistently among the most accurate simulators in terms of energy use. At the beginning until around the 4 minute mark, *OpenDC* with non-constant power models, *SimGrid* and *DISSECT-CF* with constant power model are most accurate. It is only within the last 1-2 minutes that *OpenDC*'s constant power model is the most accurate.

In both Figure 5 and Figure 7, we can see that the degree by which the simulators produce incorrect energy usage values is greater for Train Ticket workloads than for those from Sock Shop. This is especially the case for *CloudSim* and *NetworkCloudSim* and suggests that these simulators perform better for simpler microservice systems than complexer ones.

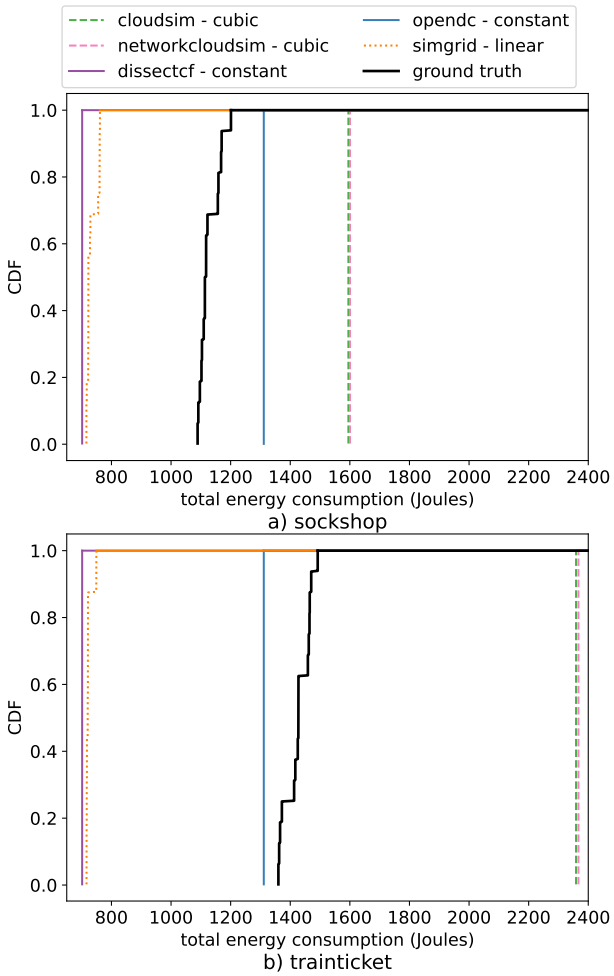



Figure 5: CDF diagrams for the total energy consumption simulated by the five simulators with their best-performing power models and the ground truth measures.

### *7.1.2. RQ1.2: Accuracy of Simulators' Power Models*

With regards to power models, we can note that constant power models perform relatively well, having the highest accuracy for *OpenDC* and *DISSECT-CF* although the two simulators have two different definitions for the constant power draw. For *CloudSim*, the cubic power model is the most accurate. For the other models, linear, square and square root, *CloudSim* has high MREs. It is notable that the *CloudSim* power models are quite similar to those used by *OpenDC*, while their simulated values are very different, especially for the square root power model. They both calculate a factor that is the difference between the energy consumption values given for maximum and idle CPU utilization at initialization and then multiply this factor with a value dependent on the power model. For the square root power model, for example, the square root of a utilization value in time to get the corresponding energy consumption value. During the calculation of this factor at the beginning, *CloudSim* also divides it by the square root of 100. In order to check that the different power models do indeed produce different energy usage values, the Kruskal-Wallis test was carried out on the power models both across simulators and for each simulator separately. These tests all resulted in a p-value of 0.0, indicating that there is indeed evidence of the power models producing significantly different energy usage values.

Finally, we can look at the standard deviation across runs for the same input trace (application, scenario,

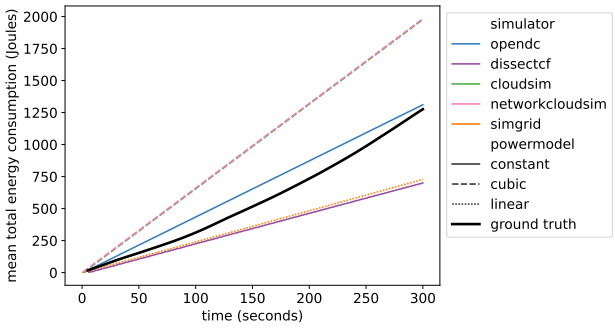



Figure 6: Mean total energy consumption in Joules for each simulator's power model that is most accurate on average.

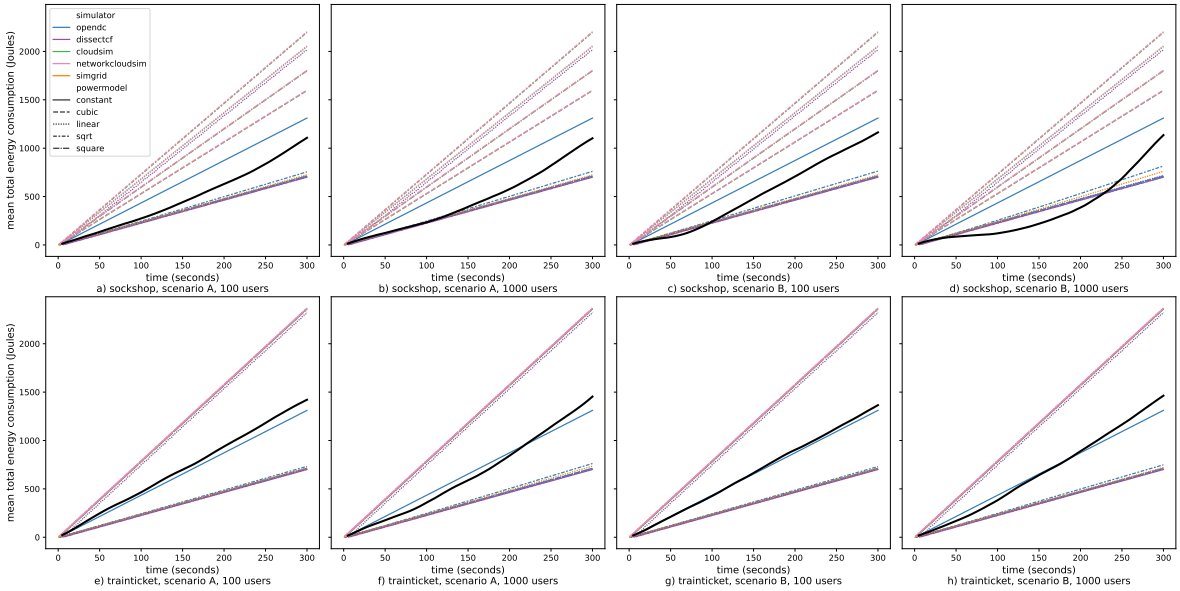



Figure 7: Mean energy usage over time for different application-scenario-users combinations.

users, repetition combination) in Table 6. There, we can see that *SimGrid* is the only simulator that seems to produce near-deterministic results.

In terms of host energy usage simulation, *OpenDC* has both the highest accuracy and the most power models to choose from. In terms of accuracy, it is followed by *SimGrid*, although this simulator only employs a linear power model. *CloudSim* offers the most power models, but only the cubic model produces simulation results comparable to those of the other well-performing models. *CloudSim* does however explicitly offer the possibility of using a self-defined power model, which could be defined to be a direct mapping from CPU utilization to power consumption values, while the other simulators do not do so.

**Energy simulation capabilities** Table 7 shows the different capabilities related to energy that the selected simulators offer. There, we can see that *SimGrid* offers the capabilities, providing extensions for the simulation of Host, Network and WiFi energy as well as Batteries, Solar Panels and Chillers (coolers). *DISSECT-CF* is the only simulator to make it possible to simulate the energy usage of storage resources, while *OpenDC* can calculate the carbon footprint of a resource given a carbon trace, as well as simulating batteries.

Table 7: Energy usage capabilities of the selected simulators.

| | *CloudSim* | *DISSECT-CF* | *NetworkCloudSim* | *OpenDC* | *SimGrid* |
|---|---|---|---|---|---|
| Host | ✓ | ✓ | ✓(custom) | ✓ | ✓ |
| Storage | ✗ | ✓ | ✗ | ✗ | ✗ |
| Network | ✗ | ✗ | ✗ | | ✓ |
| WiFi | ✗ | ✗ | ✗ | ✗ | ✓ |
| Battery | ✗ | ✗ | ✗ | ✓ | ✓ |
| Solar Panel | ✗ | ✗ | ✗ | ✗ | ✓ |
| Chiller | ✗ | ✗ | ✗ | ✗ | ✓ |
| Carbon trace | ✗ | ✗ | ✗ | ✓ | ✗ |

Table 8: System topology parameters of the selected simulators.

| **Resource** | **Property** | **CloudSim** | **DISSECT-CF** | **NetworkCloudSim** | **OpenDC** | **SimGrid** |
|---|---|---|---|---|---|---|
| Host | CPU speed | MIPS | instructions per tick | MIPS | MHz | FLOPS |
| | Memory | bytes | bytes | bytes | bytes | ✗ |
| | Network bandwidth | B/s | bytes | B/s | ✗ | ✗ |
| | Network latency | ✗ | ✗ | ✗ | ✗ | ✗ |
| Storage | Size | ✗ | MB | ✗ | ✗ | ✗ |
| | Network bandwidth | ✗ | B/s | ✗ | ✗ | ✗ |
| | Disk bandwidth | ✗ | B/s | ✗ | ✗ | ✗ |
| Network link | Bandwidth | ✗ | B/s | B/s | ✗ | bps |
| | Latency | ✗ | ticks | seconds | ✗ | ms |
| Definition method | | in code | in code | in code | JSON | XML & in code |

### *7.2. RQ2: Suitability to Simulating Microservices*

#### *7.2.1. RQ2.2 Configurability*

While the available models and extents to which the simulators support simulation of energy usage have been discussed in the previous subsection 7.1, we will now look at the configurability of the selected simulators. In Table 8, we can see the different parameters through which a system topology is defined for each of the simulators. There, we can see that *DISSECT-CF* offers the most parameters to be set by the user. While this makes the simulator highly customizable, it also means that it is more complex to define a topology. The other simulators offer relatively simple topology definitions, although there are also different degrees of customizability here. For *CloudSim* and *NetworkSim* a lot of policies such as VM schedulers or host allocation policies have to be defined for a simulation. Some simple policies are already provided and the user can also define their own. *SimGrid* and *OpenDC* provide the most simplistic ways of definiting the system topology, through XML and JSON files, respectively. *SimGrid* does also provide the possibility of defining systems in code. The manner through which *CloudSim*, *NetworkCloudSim*, *DISSECT-CF* and *SimGrid* define the host's CPU speed is also notable, as the use variations of the Instruction Per Cycle metric which is not just dependent on the hardware system, but also on factors such as the operating system and the application(s) being run [48]. This is not a constant value and must be measured for each specific context and, while it may lead to more accurate results, it is also more complex to obtain.

Table 9: Workload definition parameters of the selected simulators.

| **Parameter** | ***CloudSim*** | ***DISSECT-CF*** | ***NetworkCloudSim*** | ***OpenDC*** | ***SimGrid*** |
|---|---|---|---|---|---|
| Trace formats | swf | gwf, swf, srtg | swf | parquet | ✗ |
| Definition in code | ✓ | ✓ | ✓ | ✗ | ✓ |
| Duration | Million Instructions | seconds | Million Instructions | milliseconds | FLOPS |
| CPU consumption | fraction | Instructions | fraction | MHz | ✗ |
| Memory consumption | fraction | kB/processor | fraction | MB | bits |
| Bandwidth consumption | fraction | bps | fraction | ✗ | ✗ |
| Message size | ✗ | bytes | bytes | ✗ | bytes |

The workload definition of each simulator is described in Table 9. As mentioned in subsection 6.4, *CloudSim* does provide functions for loading swf traces, this functionality is flawed as it disregards the *submit time* field, thereby causing all jobs to be submitted at the very start of the simulation. *DISSECT-CF*s trace reader is more functional and can take multiple trace formats. *OpenDC* instead works with two parquet files per workload. One to define the jobs on a higher level and one to provide CPU usage snapshots. This is more suited towards microservice applications, as we can treat the services as jobs to be defined in the file with higher abstraction and give CPU utilization periodically in the other file. Memory utilization is only defined on a job-level and not over time, however, making it less precise in that regard. Furthermore, for *OpenDC*, workloads cannot be defined in code. For *SimGrid*, workloads can be defined in code, meaning that trace loading capabilities have to be implemented by the user themselves. It does also have functions for loading DAX[11] or DOT files, but only in the context of Directed Acyclic Graphs (DAGs), which is not relevant to microservice systems. Workloads in *SimGrid* only consist of computation duration, memory consumption or message size, with each consumption having to be defined separately. *CloudSim*, *NetworkCloudSim* and *DISSECT-CF* may seem similar in their workload definitions, but are fundamentally different. *DISSECT-CF* is a discrete simulator, meaning that the time unit it simulates is user-defined, as the simulator itself works with *ticks*. Furthermore, the concept of communication employed by *DISSECT-CF* is different from that of the other simulators that offer communication features, as *DISSECT-CF* treats communication only as resource consumption and not messages being passed back and forth.

#### *7.2.2. RQ2.1 Ease of Use*

**Simulation Process** Out of the tested simulators, only *OpenDC* has a GUI, in the form of a web application[12]. This GUI is very limited however, as it only gives the user the opportunity to choose from predefined topology elements such as CPU, GPU, memory, etc. Furthermore, only one predefined workload can be simulated. This makes the GUI unsuitable to simulating microservice systems. While *CloudSim* does not provide a GUI itself, there are a number of simulators built on the *CloudSim* engine that do such as *CloudReports* [49] and *CloudAnalyst* [50].

For all five simulators, it is possible to retrieve the consumed energy with a user-defined frequency (e.g. every second). For *CloudSim* and *NetworkCloudSim*, the power consumption per host and per datacenter is logged, and the total consumed energy in Joules is written to a csv file. The sample frequency is given in seconds, meaning that the maximum sampling frequency is one second. Terminal outputs and logs are largely user-defined (see Figure 10). For *DISSECT-CF*, the user has to implements energy monitoring with using given `EnergyMeter` classes. As *DISSECT-CF* is an discrete event simulator, its time granularity is user-defined. Terminal outputs and logs are also largely user-defined (see Figure 11). *OpenDC* writes the power consumption in W of hosts to parquet files with a maximum sampling frequency of one second. The terminal output, on the other hand, is very limited (see Figure 12). Finally, for *SimGrid*, total energy consumption per host can be retrieved, but this has to be implemented by the user themselves in code (for example terminal output see Figure 13).

**Documentation** As indicated above, *CloudSim* is a very widely used cloud simulator and therefore has a relatively wide range of documentation and examples available. The documentation provided in the *CloudSim* distribution is generally up to date but not very descriptive, for example lacking units of variables in the in-code documentation such as storage size or bandwidth. The same is the case for *NetworkCloudSim*. For the power and network capabilities of *CloudSim*, the documentation is less extensive. Documentation of *DISSECT-CF* is quite little, but an accompanying repository with examples[13] is available that was also used to implement the simulations for this study. Next to in-code documentation, both *OpenDC*[14] and *SimGrid*[15] have external documentation available. Both are very clear and thorough, but *SimGrid's* is significantly more extensive and up-to-date. For all five discussed simulators, some form of documentation is available

[11] https://pegasus.isi.edu/documentation/development/schemas.html
[12] https://opendc.org/projects
[13] https://github.com/kecskemeti/dissect-cf-examples
[14] https://atlarge-research.github.io/opendc/docs/intro
[15] https://simgrid.org/doc/latest

and minimally sufficient, although the lack of clear and extensive documentation of *DISSECT-CF* could increase the steepness of its learning curve.

Table 10: Simulation time statistics for the selected simulators in seconds.

| Simulator | Mean | Median | Std | Min | Max |
|---|---|---|---|---|---|
| CloudSim | 3.14 | 3.13 | 0.58 | 2.24 | 4.17 |
| DISSECT-CF | 2.75 | 2.75 | 0.09 | 2.42 | 3.15 |
| NetworkCloudSim | 3.70 | 3.71 | 1.00 | 2.39 | 5.15 |
| OpenDC | 0.50 | 0.50 | 0.01 | 0.47 | 0.54 |
| SimGrid | 0.05 | 0.04 | 0.03 | 0.01 | 0.16 |
| SimGrid with comm. | 0.67 | 0.26 | 0.73 | 0.07 | 2.35 |

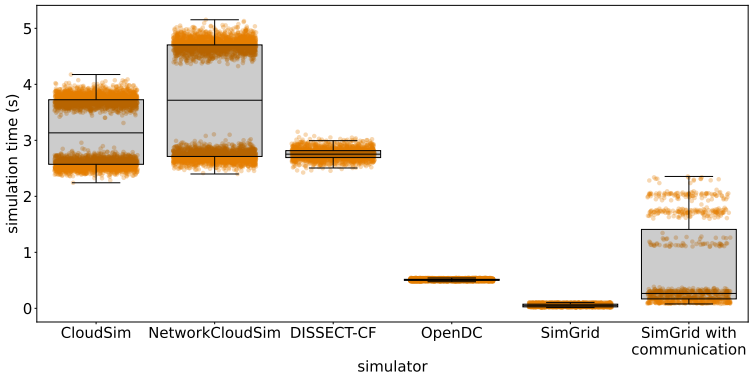



Figure 8: Combination of a box- and jitter-plot of the simulation times in seconds of the simulated workloads for the different simulators.

**Simulation Time** As can be seen in Table 10 and Figure 8, *NetworkCloudSim*, *CloudSim* and *DISSECT-CF* have similar mean run times between 2.75 and 3.75 seconds, with *DISSECT-CF* being the more consistent. The mean simulation times for these simulators stay below 4 seconds on average, making these simulators well-suited to simulating small to medium-sized microservice applications. *OpenDC* and *SimGrid* with and without communication occupy the next bracket with mean run times between 0.05 and 0.67 seconds, allowing larger microservice applications to be simulated.

One notable observation is that when communication is simulated, it can have a significant impact on runtime. This is the case for both *SimGrid* with communication and *NetworkCloudSim*. In the case of *NetworkCloudSim* two distinct groups can be seen in Figure 8; this would be the distinction between the two services simulated, as Train Ticket has more services than Sock Shop. A similar division is seen in the *CloudSim* results, but with a smaller difference, indicating this is not solely a product of simulating networks. *SimGrid* with communication also displays grouping behaviour but has a larger number of groups, with the majority of cases still having a runtime similar to the base SimGrid run. In contrast with the CloudSim design, this is a product of simulating more users. As users increase CloudSim with communication displays a larger difference between groups, this phenomenon is present for both simulated services. As this behavior is not present in the base SimGrid design, it can be concluded that this is a product of simulating network communication. This indicates that choosing the best simulator in terms of runtime can depend largely on the simulation target; number of users, microservices, and the presence of network simulation can all influence the runtime of most simulators. However, in general, OpenDC remains the most consistent in terms of runtime across these factors.

## 8. Discussion

In Section 7.1.1, we could observe clear differences between the accuracy of different simulators and power models. Although *OpenDC* and *DISSECT-CF* use different values as the constant power consumption value,

both their constant models performed quite well. For their constant values, *OpenDC* a user-defined value while *DISSECT-CF* uses the idle power consumption value. A reason for the good accuracy of the constant models performing well could be the fact that the power usage values given to these models were derived from the superset that contains the same traces they then simulated. This suggests that these models are good at modelling stable, predictable behaviour, but perhaps this accuracy does not carry over to more unpredictable system or applications. Between the *Sock Shop* and *Train Ticket* traces, we can see that with the higher complexity of *Train Ticket*, simulators produce more similar results, also across power models. *CloudSim*, *NetworkCloudSim* and *DISSECT-CF* with the linear power model drastically overestimate the power usage, while *DISSECT-CF* with the constant power model, *SimGrid* and *OpenDC* with the non-constant power models underestimate the power usage. An explanation for this could be that *Train Ticket* has more services than *Sock Shop* in combination with the CPU usage value being given per service. If there is a per-service margin of error and a larger number of services, the total error increases with a larger number of service. As microservice systems in industry can have many more services than *Train Ticket*, and it is precisely these systems that consume the largest amounts of energy, this indicates a shortcoming of the simulators towards simulating microservice systems and one of the areas for improvement in the future regarding cloud simulators.

### *8.1. Future Work*

- **Simulators calibration**: as we will discuss in Section 9.1, for all simulators we configured the hardware-dependent idle and maximum power consumption values of the host machines, while we kept all the other simulation parameters to their default values. An important direction for future research is the systematic investigation of the impact of simulator calibration on energy-estimation accuracy and how different calibration strategies affect the predictive performance of different simulators across diverse workloads, infrastructures, and/or deployment scenarios. Building on such evidence, it would be possible to develop a decision model that supports researchers and practitioners in (i) selecting the most suitable cloud simulator for a given energy-evaluation objective and also (ii) identifying appropriate calibration strategies for that simulator. Such a decision model could provide guidelines on the expected trade-offs among calibration effort, simulation complexity, reproducibility, and estimation accuracy, ultimately leading to more accurate energy evaluations of microservice systems.

- **Dynamic Power models**: as shown in subsection 7.1, the power model that best describes the energy consumption of a microservice system can change over time. Therefore, research into dynamic power models that change depending on the context could be an interesting area of improvement for energy simulation of microservice systems.

- **Service-level energy simulation**: energy simulation in cloud simulators is currently only available on a host-level. This makes the use of these simulators for microservice research limited, making it very hard to carry out root cause analysis into extraordinarily high energy usage levels caused by certain parts of the microservice system, for example. Assessing service-level energy-consumption estimation remains an important future research direction for researchers working on cloud simulators and would require suitable fine-grained power models for individual services and publicly available datasets providing reliable per-service energy measurements (the ground truth dataset we constructed for this study is a first step towards this direction).

- **Scalability**: the simulators studied have scalability issues both in terms of accuracy decreasing with increased system complexity and simulation time increasing when communication is simulated. The limited scalability of cloud simulators currently available has been noted by others (e.g. [24]) and is particularly relevant to the microservices context, as microservice systems have a lot of communication and are often complex systems comprised of many services. Furthermore, it is especially the big, complex microservice systems for which the energy usage should be reduced.

- **Increased Usability**: often, cloud simulators are a tool used for achieving a certain goal instead of the goal themselves. Out of the 5 simulators discussed here, none have a GUI usable in a research context,

and for all except *OpenDC*, users have to largely code the experiment to be simulated themselves. Although documentation or examples are available, this still means that simulation takes considerable effort and users have to be able to code in the respective simulator's language. The threshold for using simulators would be decreased if they were equipped with a usable GUI. Furthermore, the ability to load traces from file could be implemented for all simulators, as well as some basic energy usage reporting such that the users do not have to implement these themselves.

## 9. Threats To Validity

In this section, we report the possible threats to the validity of the results and conclusions presented in this study according to the classification framework proposed by Wohlin *et al.* [31].

### *9.1. Internal Validity*

Internal validity can be threatened through influence by external factors. For the ground truth collection, this threat was mitigated as much as possible through random treatment selection to mitigate order-related bias. The threat of carryover effects from one run affecting the next was mitigated by adding warm-up and cooldown periods between runs. For the simulator testing, order-related bias and carryover effects are only a threat to the measuring of the mean simulation time. These threats were mitigated by having a large sample size of simulation runs and simulating one repetition per treatment with all simulators before repeating with another repetition.

Although Scaphandre is a widely used energy monitoring tool, it can cause data inconsistencies. Because of this, each treatment was run 30 times and the four repetitions with the energy consumption measures closest to the mean of these 30 repetitions were selected as traces to be simulated. As shown in subsubsection 7.1.1, some of the tested simulators are non-deterministic. In order to get an indication of the general performance of these simulators, every simulation of a workload was also run 30 times and the average over these 30 runs was taken.

We did not perform simulator-specific parameter calibration beyond configuring the hardware-dependent idle and maximum power consumption values required by the simulators. Introducing simulator-specific tuning would have raised additional threats to internal validity. In particular, the calibration process would have been performed by the authors, potentially introducing experimenter bias through subjective choices regarding parameter values and optimization strategies. Moreover, the simulators expose substantially different energy models and abstractions, resulting in large and heterogeneous calibration design spaces. Defining a representative and comparable calibration strategy across all simulators would therefore be challenging and could itself influence the observed results. For these reasons, we opted to evaluate each simulator using its default configuration and built-in power models, thereby ensuring a fair and reproducible comparison of their out-of-the-box capabilities.

Evaluated simulators have heterogeneous support for communication modeling. While NetworkCloudSim and SimGrid explicitly model network communication and message exchanges, the other simulators provide limited or no communication-simulation capabilities. Since communication activities can affect resource utilization and, consequently, host-level energy consumption, differences in communication modeling may have influenced our results. Future work should explicitly assess the impact of communication simulation on energy-estimation accuracy.

### *9.2. External Validity*

External validity concerns the generalizability of the results discussed in this study to other systems and applications. Both *Sock Shop* and *Train Ticket* are microservice applications that are widely used in research [51] (e.g., [52, 53, 54, 11, 55]) and are of varying complexity. They are, however, academic benchmarking systems and may not be representative of other microservice systems that are more complex, have dynamic workloads or run on other technologies. Furthermore, the systems that were used to run the applications on were relatively small, which may not represent larger-scale microservice deployments. Finally, in this study we only look at the host energy usage of the cloud systems microservices run on. While this forms the

biggest energy usage of microservices and other software systems, it does not cover the whole scope. Servers (including storage) are estimated to use 50% of energy in a data centre. For networks, this is 8% [19]. As the other components such as cooling systems, UPS and PDU were not part of the simulations carried out for this study or the experiments to obtain the ground truth meaures, this means that approximately 42% of energy usage of data centres was not taken into account. *SimGrid* is the only simulator discussed here that has an option for modelling cooling systems with the Chiller plug-in, but this plug-in was not studied. Because of this, the accuracy results presented in this study can not be generalized to the suitability of these simulators for simulating overall energy usage of microservice applications, but only the energy usage of the application hosts.

### *9.3. Construct Validity*

Construct validity is threatened by the executing of the experiment influencing the results. To mitigate this threat, the metric collection during the ground truth collection phase was done on a separate server. The monitoring tools such as Scaphandre and cAdvisor may have had an influence on the overall measured host power, but as all system properties such as idle power usage, maximum power usage and IPC were collected with these systems running as well, the influence should be minimized. Another threat is that for each metric only one tool was used, Scaphandre is widely used in research and peer-reviewed (e.g., [56, 18, 57, 58, 59]), while cAdvisor is widely adopted in both research and industry. In order to mitigate the threat of the initial collection of cloud simulators not being broad enough and missing, for example, recently published simulators, a research practitioner from a relevant research background was consulted as a source for simulators as well.

### *9.4. Conclusion Validity*

Conclusion validity concerns the reliability of the conclusions drawn in this study. To ensure this, all measurements were taken with 30 repetitions to increase the reliability of the results. Mean relative error was used as a metric for accuracy as it is widely used to evaluate the accuracy of scalar values while keeping the metric in the context of the domain. Whether there is evidence of significant differences between the simulators and power models was determined using the Mann-Whitney U and Kruskal-Wallis tests. For the second research question concerning the usability of the simulators, there is a threat to the validity of the results as these were only collected by one person. In order to mitigate this threat, the maintainers of the simulators were contacted and asked to confirm those results. However, only the maintainers for *CloudSim*, *NetworkCloudSim* and *SimGrid* responded. Their feedback was taken into account when revising the paper.

## 10. Conclusion

This study evaluates and discusses five cloud simulators, *CloudSim*, *DISSECT-CF*, *NetworkCloudSim*, *OpenDC* and *SimGrid* for their accuracy and general suitability to simulating the energy consumption of microservice systems. The microservice systems *Sock Shop* and *Train Ticket* are simulated and the simulated energy usage values are compared to real-life measures. This is done with the goal of enabling researchers and other prospective users to make well-informed decisions regarding what simulator to choose for their specific context. In order to simplify this decision process, an overview of the simulators' properties discussed in this study is shown in Figure 9. From this, *OpenDC* emerges as the simulator able to obtain the most accurate results, followed by *SimGrid*, *DISSECT-CF* and finally *CloudSim* and *NetworkCloudSim*. From the selected simulators, *SimGrid* has the most extensive energy-related capabilities. *OpenDC* is the most simple to use, while *DISSECT-CF* is the most customizable simulator. *DISSECT-CF* is the only discrete simulator in the selection, however, which, combined with only very little documentation, can lead to a higher learning curve. With regards to simulation time, all five simulators take on average less than 4 seconds per simulation to run. *SimGrid* is on average the fastest with a mean runtime of 0.05, dropping to third with a mean runtime of 0.67 seconds when communication is simulated.

With regards to future research, this study opens up some further avenues of interest (see Section 8). As this research focuses only on the simulation of host energy usage, in the future energy usage across

microservice systems at a larger scale with dynamic workloads could be used to assess these simulators. Moreover, the simulators in these study were not calibrated, *i.e.,* the simulator parameters were not adjusted from the initial values to produce realistic results [60]. While this can be a complex process, it can be important in contexts where precise and fine-grained results should be obtained. A similar study including a further step of calibration could therefore be valuable. Furthermore, some of the results in this study could be used to improve the simulators discussed here, for example through dynamic power model selection, or even by combining successful features from different simulators. Finally, the results presented in this study should be used to choose the right simulators for specific contexts and utilize them to develop solutions to systematically reduce the host-level energy usage of microservice systems.

## References


[1] IEA, Energy and AI, World Energy Outlook Special Report, IEA, Paris, France (Apr. 2025).
URL www.iea.org/reports/energy-and-ai

[2] M. A. B. Siddik, A. Shehabi, L. Marston, The environmental footprint of data centers in the United States, Environmental Research Letters 16 (6) (2021) 064017, publisher: IOP Publishing. doi:10.1088/1748-9326/abfba1.
URL https://dx.doi.org/10.1088/1748-9326/abfba1

[3] S. G. Monserrate, The Cloud Is Material: On the Environmental Impacts of Computation and Data Storage, MIT Case Studies in Social and Ethical Responsibilities of Computing Winter 2022 (2022) 2–33.

[4] Statista, Global usage of microservices in organizations 2021, by organization size, Tech. rep., Statista (Apr. 2021).
URL https://www.statista.com/statistics/1236823/microservices-usage-per-organization-size/

[5] Gartner Peer Community, Microservices Architecture: Have Engineering Organizations Found Success?, Tech. rep., Gartner (Oct. 2023).
URL https://www.gartner.com/peer-community/oneminuteinsights/omi-microservices-architecture-have-engineering-organizations-found-success-u6b

[6] M. Loukides, S. Swoyer, Microservices Adoption in 2020, Tech. rep., O'Reilly (Jul. 2020).
URL https://www.oreilly.com/radar/microservices-adoption-in-2020/

[7] G. Araújo, V. Barbosa, L. N. Lima, A. Sabino, C. Brito, I. Fé, P. Rego, E. Choi, D. Min, T. A. Nguyen, F. A. Silva, Energy Consumption in Microservices Architectures: A Systematic Literature Review, IEEE Access 12 (2024) 186710–186729. doi:10.1109/ACCESS.2024.3389064.
URL https://ieeexplore.ieee.org/document/10500420

[8] M. Pirhonen, K. Systä, D. Hästbacka, The Pains and Gains of Microservices Revisited, in: D. Pfahl, J. Gonzalez Huerta, J. Klünder, H. Anwar (Eds.), Product-Focused Software Process Improvement, Springer Nature Switzerland, Cham, 2025, pp. 238–254. doi:10.1007/978-3-031-78386-9_16.

[9] A. Katal, S. Dahiya, T. Choudhury, Energy efficiency in cloud computing data centers: a survey on software technologies, Cluster Computing 26 (3) (2023) 1845–1875. doi:10.1007/s10586-022-03713-0.
URL https://doi.org/10.1007/s10586-022-03713-0

[10] N. Mansouri, R. Ghafari, B. M. H. Zade, Cloud computing simulators: A comprehensive review, Simulation Modelling Practice and Theory 104 (2020) 102144. doi:10.1016/j.simpat.2020.102144.

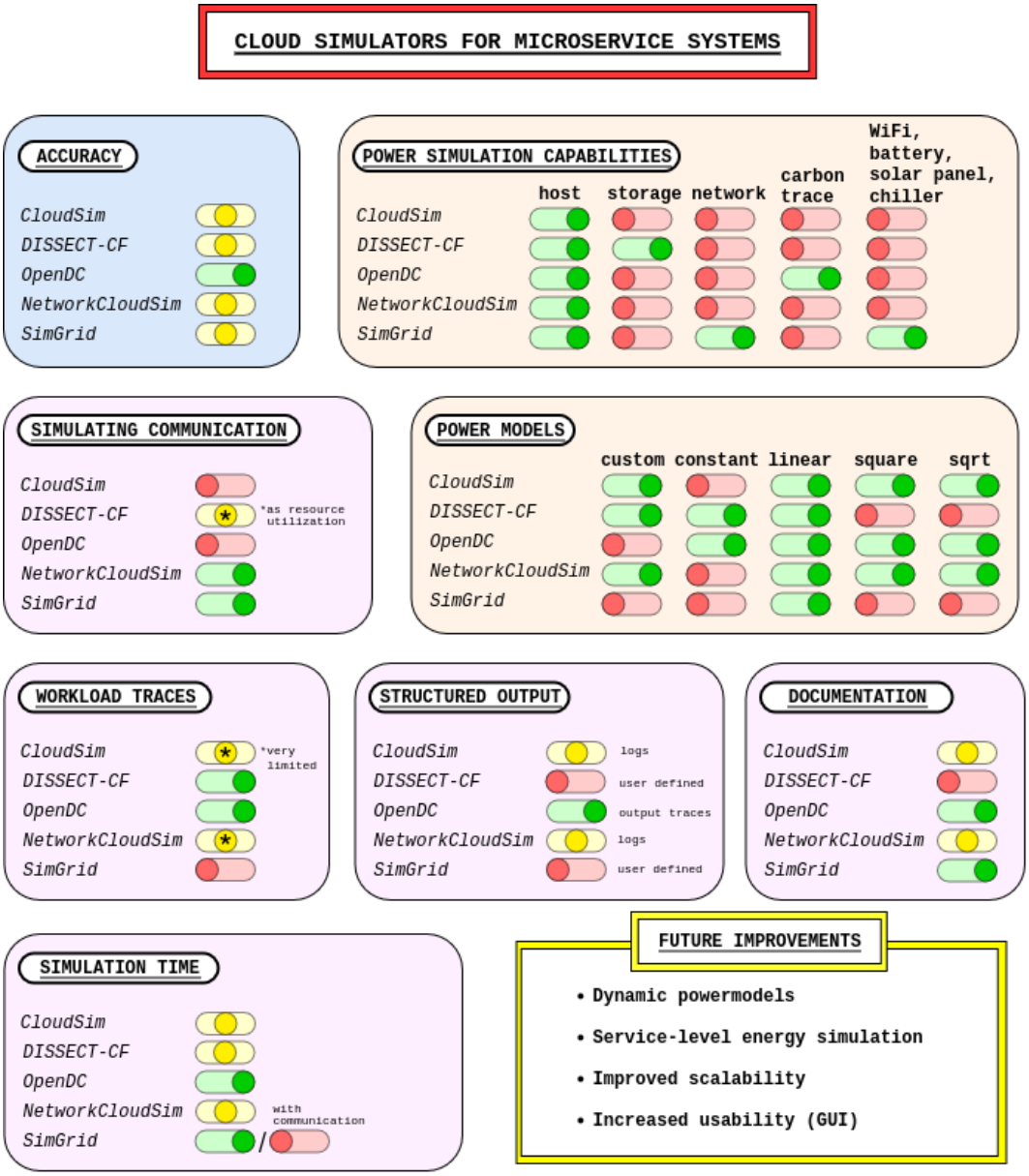



Figure 9: Graphic giving an overview of the discussed simulators' properties.

[11] X. Zhou, X. Peng, T. Xie, J. Sun, C. Xu, C. Ji, W. Zhao, Benchmarking microservice systems for software engineering research, in: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings, ICSE '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 323–324. doi:10.1145/3183440.3194991.
URL https://dl.acm.org/doi/10.1145/3183440.3194991

[12] M. Mazzara, K. Khanda, R. Mustafin, V. Rivera, L. Safina, A. Sillitti, Microservices Science and Engineering, in: P. Ciancarini, S. Litvinov, A. Messina, A. Sillitti, G. Succi (Eds.), Proceedings of 5th International Conference in Software Engineering for Defence Applications, Springer International Publishing, Cham, 2018, pp. 11–20. doi:10.1007/978-3-319-70578-1_2.

[13] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, Today, and Tomorrow, in: M. Mazzara, B. Meyer (Eds.), Present and Ulterior Software Engineering, Springer International Publishing, Cham, 2017, pp. 195–216. doi:10.1007/978-3-319-67425-4_12.
URL https://doi.org/10.1007/978-3-319-67425-4_12

[14] R. Verdecchia, P. Lago, C. Ebert, C. de Vries, Green IT and Green Software, IEEE Software 38 (6) (2021) 7–15. doi:10.1109/MS.2021.3102254.
URL https://ieeexplore.ieee.org/document/9585139

[15] X. Xiao, Architectural Tactics to Improve the Environmental Sustainability of Microservices: A Rapid Review, arXiv:2407.16706 [cs] (Jul. 2024). doi:10.48550/arXiv.2407.16706.
URL https://doi.org/10.48550/arXiv.2407.16706

[16] A. Noureddine, R. Rouvoy, L. Seinturier, A review of energy measurement approaches, SIGOPS Oper. Syst. Rev. 47 (3) (2013) 42–49. doi:10.1145/2553070.2553077.
URL https://dl.acm.org/doi/10.1145/2553070.2553077

[17] E. O'Dea, A Measurement-Driven Approach to Enhancing Sustainability in Microservice Architectures, in: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), IEEE, Odense, Denmark, 2025, pp. 167–172, iSSN: 2768-4288. doi:10.1109/ICSA-C65153.2025.00031.
URL https://ieeexplore.ieee.org/abstract/document/11014885

[18] M. Jay, V. Ostapenco, L. Lefevre, D. Trystram, A.-C. Orgerie, B. Fichel, An experimental comparison of software-based power meters: focus on CPU and GPU, in: 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid), IEEE, Bangalore, India, 2023, pp. 106–118. doi:10.1109/CCGrid57682.2023.00020.
URL https://ieeexplore.ieee.org/document/10171575

[19] R. Buyya, S. Ilager, P. Arroba, Energy-efficiency and sustainability in new generation cloud computing: A vision and directions for integrated management of data centre resources and workloads, Software: Practice and Experience 54 (1) (2024) 24–38. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.3248, doi:https://doi.org/10.1002/spe.3248.
URL https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.3248

[20] A. Ahmed, A. S. Sabyasachi, Cloud computing simulators: A detailed survey and future direction, in: 2014 IEEE International Advance Computing Conference (IACC), IEEE, Gurgaon, India, 2014, pp. 866–872. doi:10.1109/IAdCC.2014.6779436.
URL https://ieeexplore.ieee.org/abstract/document/6779436

[21] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, M. M. Freire, CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness, in: 2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), IEEE, Lisbon, Portugal, 2017, pp. 400–406. doi:10.23919/INM.2017.7987304.
URL https://ieeexplore.ieee.org/abstract/document/7987304

[22] D. Kliazovich, P. Bouvry, S. U. Khan, GreenCloud: a packet-level simulator of energy-aware cloud computing data centers, The Journal of Supercomputing 62 (3) (2010) 1263–1283. doi:10.1007/s11227-010-0504-1.
URL https://doi.org/10.1007/s11227-010-0504-1

[23] A. Ismail, Energy-driven cloud simulation: existing surveys, simulation supports, impacts and challenges, Cluster Computing 23 (4) (2020) 3039–3055. doi:10.1007/s10586-020-03068-4.

[24] V. E. Martone, M. Mastroianni, F. Palmieri, Evaluation of cloud simulators for energy-related applications, in: 2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), IEEE, Sakheer, Bahrain, 2023, pp. 142–147, iSSN: 2770-7466. doi:10.1109/3ICT60104.2023.10391467.

[25] H. Rjeib, G. Kecskemeti, Towards Accurate Energy Measurement Simulations in Cloud Computing Environments, International Journal of Mathematics, Statistics, and Computer Science 3 (2025) 338–344. doi:10.59543/ijmscs.v3i.14191.

[26] V. R. Basili, G. Caldiera, H. D. Rombach, Goal question metric approach, in: Encyclopedia of Software Engineering, Vol. 1, John Wiley & Sons, New York City, United States, 1994, pp. 528–532.

[27] Weaveworks, Sock Shop (Jun. 2025).
URL https://github.com/microservices-demo/microservices-demo

[28] CodeWisdom, Train Ticket (Jun. 2025).
URL https://github.com/FudanSELab/train-ticket

[29] M. S. Floroiu, S. Russo, L. Giamattei, A. Guerriero, I. Malavolta, R. Pietrantuono, Anomaly Detection and Root Cause Analysis of Microservices Energy Consumption, in: 2024 IEEE International Conference on Web Services (ICWS), IEEE, Shenzhen, China, 2024, pp. 590–600, iSSN: 2836-3868. doi:10.1109/ICWS62655.2024.00079.
URL https://ieeexplore.ieee.org/document/10707452

[30] W. Rao, H. Li, Energy-aware Scheduling Algorithm for Microservices in Kubernetes Clouds, Journal of Grid Computing 23 (1) (2025) 1–22. doi:10.1007/s10723-024-09788-w.
URL https://link.springer.com/article/10.1007/s10723-024-09788-w

[31] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer, Berlin, Heidelberg, 2012.
URL https://doi.org/10.1007/978-3-642-29044-2

[32] Cloudslab, CloudSim (2025).
URL https://github.com/Cloudslab/cloudsim

[33] G. Kecskemeti, DISSECT-CF (2023).
URL https://github.com/kecskemeti/dissect-cf

[34] F. Mastenbroek, G. Andreadis, S. Jounaid, W. Lai, J. Burley, J. Bosch, E. van Eyk, L. Versluis, V. van Beek, A. Iosup, OpenDC 2.0: Convenient Modeling and Simulation of Emerging Technologies in Cloud Datacenters, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), IEEE, Melbourne, Australia, 2021, pp. 455–464. doi:10.1109/CCGrid51090.2021.00055.
URL https://ieeexplore.ieee.org/abstract/document/9499454

[35] simgrid, Simgrid (Jun. 2025).
URL https://github.com/simgrid/simgrid

[36] R. Buyya, R. Ranjan, R. N. Calheiros, Modeling and simulation of scalable Cloud computing environments and the CloudSim toolkit: Challenges and opportunities, in: 2009 International Conference on High Performance Computing & Simulation, IEEE, Leipzig, Germany, 2009, pp. 1–11. doi:10.1109/HPCSIM.2009.5192685.
URL https://ieeexplore.ieee.org/abstract/document/5192685

[37] F. Howell, R. McNab, SimJava: A discrete event simulation library for java, in: First International Conference on Web-based Modelling and Simulation, Society for Computer Simulation, San Diego, CA, USA, 1998, pp. 1–1.
URL https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=44e63334c630f9bc57185a1065b58e53ef3a5f47

[38] R. Buyya, M. Murshed, GridSim: a toolkit for the modeling and simulation of distributed resource management and scheduling for Grid computing - Buyya - 2002 - Concurrency and Computation: Practice and Experience - Wiley Online Library, Concurrency and Computation: Practice and Experience 14 (13-15) (2002) 1175–1220. doi:10.1002/cpe.710.
URL https://onlinelibrary.wiley.com/doi/10.1002/cpe.710

[39] S. J. Chapin, W. Cirne, D. G. Feitelson, J. P. Jones, S. T. Leutenegger, U. Schwiegelshohn, W. Smith, D. Talby, Benchmarks and Standards for the Evaluation of Parallel Job Schedulers, in: D. G. Feitelson, L. Rudolph (Eds.), Job Scheduling Strategies for Parallel Processing, Springer, Berlin, Heidelberg, 1999, pp. 67–90. doi:10.1007/3-540-47954-6_4.

[40] A. Beloglazov, R. Buyya, Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in Cloud data centers, Concurrency and Computation: Practice and Experience 24 (13) (2012) 1397–1420. doi:10.1002/cpe.1867.
URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.1867

[41] G. Kecskemeti, DISSECT-CF: A simulator to foster energy-aware scheduling in infrastructure clouds, Simulation Modelling Practice and Theory 58 (2015) 188–218. doi:10.1016/j.simpat.2015.05.009.
URL https://www.sciencedirect.com/science/article/pii/S1569190X15000842

[42] S. K. Garg, R. Buyya, NetworkCloudSim: Modelling Parallel Applications in Cloud Simulations, in: 2011 Fourth IEEE International Conference on Utility and Cloud Computing, IEEE, Melbourne, VIC, Australia, 2011, pp. 105–113. doi:10.1109/UCC.2011.24.
URL https://ieeexplore.ieee.org/document/6123487

[43] H. Casanova, A. Giersch, A. Legrand, M. Quinson, F. Suter, Lowering entry barriers to developing custom simulators of distributed applications and platforms with SimGrid, Parallel Computing 123 (2025) 103–125. doi:https://doi.org/10.1016/j.parco.2025.103125.

[44] H. Casanova, Simgrid: a toolkit for the simulation of application scheduling, in: Proceedings First IEEE/ACM International Symposium on Cluster Computing and the Grid, IEEE, Brisbane, QLD, Australia, 2001, pp. 430–437. doi:10.1109/CCGRID.2001.923223.
URL https://ieeexplore.ieee.org/document/923223

[45] F. C. Heinrich, T. Cornebize, A. Degomme, A. Legrand, A. Carpen-Amarie, S. Hunold, A.-C. Orgerie, M. Quinson, Predicting the Energy Consumption of MPI Applications at Scale Using a Single Node, in: NOMS 2024-2024 IEEE Network Operations and Management Symposium, IEEE, Hawaii, United States, 2017, pp. 1–3.
URL https://inria.hal.science/hal-01523608

[46] B. Rodriguez Sanchez, Multivariate Anomaly Detection and Root Cause Analysis of Energy Issues in Microservice-based Systems, Master's thesis, University of Amsterdam and Vrije Universiteit Amsterdam, Amsterdam (Dec. 2024).
URL https://github.com/bertars/Thesis

[47] S. Vegas, C. Apa, N. Juristo, Crossover Designs in Software Engineering Experiments: Benefits and Perils, IEEE Transactions on Software Engineering 42 (2) (2016) 120–135. doi:10.1109/TSE.2015.2467378.
URL https://ieeexplore.ieee.org/document/7192651

[48] D. A. Patterson, J. L. Hennessy, Parallelism via Instructions, in: Computer Organization and Design: The Hardware/Software Interface, 5th Edition, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2013, pp. 332–344.

[49] T. Teixeira Sá, R. N. Calheiros, D. G. Gomes, CloudReports: An Extensible Simulation Tool for Energy-Aware Cloud Computing Environments, in: Z. Mahmood (Ed.), Cloud Computing: Challenges, Limitations and R&D Solutions, Springer International Publishing, Cham, 2014, pp. 127–142. doi:10.1007/978-3-319-10530-7_6.
URL https://doi.org/10.1007/978-3-319-10530-7_6

[50] B. Wickremasinghe, R. N. Calheiros, R. Buyya, CloudAnalyst: A CloudSim-Based Visual Modeller for Analysing Cloud Computing Environments and Applications, in: 2010 24th IEEE International Conference on Advanced Information Networking and Applications, IEEE, Perth, WA, Australia, 2010, pp. 446–452, iSSN: 2332-5658. doi:10.1109/AINA.2010.32.
URL https://ieeexplore.ieee.org/document/5474733

[51] S. Fischer, P. Urbanke, R. Ramler, M. Steidl, M. Felderer, An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study, in: Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), AST '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 182–192. doi:10.1145/3644032.3644445.
URL https://dl.acm.org/doi/10.1145/3644032.3644445

[52] A. U. Gias, G. Casale, M. Woodside, ATOM: Model-Driven Autoscaling for Microservices, in: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), IEEE, Dallas, TX, USA, 2019, pp. 1994–2004, iSSN: 2575-8411. doi:10.1109/ICDCS.2019.00197.
URL https://ieeexplore.ieee.org/abstract/document/8884900

[53] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root Cause Localization of Performance Issues in Microservices, in: NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium, IEEE, Budapest, Hungary, 2020, pp. 1–9, iSSN: 2374-9709. doi:10.1109/NOMS47738.2020.9110353.

[54] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, M. Kocaoglu, Root Cause Analysis of Failures in Microservices through Causal Discovery, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Advances in Neural Information Processing Systems, Vol. 35, Curran Associates, Inc., New Orleans, LA, USA, 2022, pp. 31158–31170.
URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c9fcd02e6445c7dfbad6986abee53d0d-Paper-Conference.pdf

[55] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: trace-log combined microservice anomaly detection through graph-based deep learning, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 623–634. doi:10.1145/3510003.3510180.
URL https://dl.acm.org/doi/10.1145/3510003.3510180

[56] C. Centofanti, J. Santos, V. Gudepu, K. Kondepu, Impact of power consumption in containerized clouds: A comprehensive analysis of open-source power measurement tools, Computer Networks 245 (2024) 110371. doi:10.1016/j.comnet.2024.110371.
URL https://www.sciencedirect.com/science/article/pii/S1389128624002032

[57] E. Cadorel, D. Saingre, A Protocol to Assess the Accuracy of Process-Level Power Models, in: 2024 IEEE International Conference on Cluster Computing (CLUSTER), IEEE, Kobe, Japan, 2024, pp. 74–84, iSSN: 2168-9253. doi:10.1109/CLUSTER59578.2024.00014.
URL https://ieeexplore.ieee.org/document/10740809

[58] N. K. Shankaranarayanan, Z. Li, I. Seskar, P. Maddala, S. Puthenpura, A. Stancu, A. Agarwal, POET: A Platform for O-RAN Energy Efficiency Testing, in: 2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall), IEEE, Washington, DC, USA, 2024, pp. 1–5, iSSN: 2577-2465. doi:10.1109/VTC2024-Fall63153.2024.10757537.
URL https://ieeexplore.ieee.org/abstract/document/10757537

[59] V. Gudepu, R. R. Tella, C. Centofanti, J. Santos, A. Marotta, K. Kondepu, Demonstrating the Energy Consumption of Radio Access Networks in Container Clouds, in: NOMS 2024-2024 IEEE Network Operations and Management Symposium, IEEE, Seoul, Korea, Republic of, 2024, pp. 1–3, iSSN: 2374-9709. doi:10.1109/NOMS59830.2024.10575134.
URL https://ieeexplore.ieee.org/abstract/document/10575134

[60] A. M. Uhrmacher, Seven pitfalls in modeling and simulation research, in: Proceedings of the Winter Simulation Conference, WSC '12, Winter Simulation Conference, Berlin, Germany, 2012, pp. 1–12.
URL https://https://dl.acm.org/doi/10.5555/2429759.2430188

## 11. Simulator Selection

Table 11: The 13 selected cloud simulator review papers used to extract cloud simulators from.

| Id | Title | Authors | Venue | Year |
|---|---|---|---|---|
| P1 | Cloud computing simulators: A comprehensive review | Najme Mansouri, Reyhane Ghafari, Behnam Mohammad, Hasani Zade, | Simulation Modelling Practice and Theory | 2020 |
| P2 | Cloud computing simulators: A detailed survey and future direction | Arif Ahmed, Abadhan Saumya Sabyasachi | International Advance Computing Conference (IACC) | 2014 |
| P3 | A Review of Cloud Computing Simulation Platforms and Related Environments | James Byrne, Sergej Svorobej, Konstantinos M. Giannoutakis, Dimitrios Tzovaras, P. J. Byrne, Per-Olov Östberg, Anna Gourinovitch, Theo Lynn | International Conference on Cloud Computing and Services Science (CLOSER) | 2017 |
| P4 | Modeling and simulation of cloud computing: A review | Wei Zhao, Yong Peng, Feng Xie, Zhonghua Dai | Asia Pacific Cloud Computing Congress (APCloudCC) | 2013 |
| P5 | A Survey on Cloud Computing Simulation and Modeling | Ailyas Bambrik | SN Computer Science | 2020 |
| P6 | Simulation tools for cloud computing: A survey and comparative study | Fairouz Fakhfakh, Hatem Hadj Kacem, Ahmed Hadj Kacem | International Conference on Computer and Information Science (ICIS) | 2017 |
| P7 | Building a cloud on earth: A study of cloud computing data center simulators | Mohamed Abu Sharkh, Ali Kanso, Abdallah Shami, Peter Öhlén | Computer Networks | 2016 |
| P8 | Energy-driven cloud simulation: existing surveys, simulation supports, impacts and challenges | Azlan Ismail | Cluster Computing | 2020 |
| P9 | Cloud Computing Simulators: A Review | Yousef Sanjalawe, Salam Al-E'mari | International Arab Conference on Information Technology (ACIT) | 2023 |
| P10 | A Comprehensive Survey on Cloud Computing Simulators | Oladosu Oyebisi Oladimeji, Dasola Oyeyiola, Olayanju Oladineji, Pelumi Oyeyiola | Scientific Journal of Informatics | 2021 |
| P11 | Evaluation of cloud simulators for energy-related applications | Vincenzo Emanuele Martone, Michele Mastroianni, Francesco Palmieri | International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT) | 2023 |
| P12 | Cloud Computing and Simulation Paradigms: A Technical Exploration and Analysis | Divya Kapil, Varsha Mittal, Atika Gupta | International Conference on Computing Communication and Networking Technologies (ICCCNT) | 2024 |
| P13 | A Systematic Parameter Analysis of Cloud Simulation Tools in Cloud Computing Environments | Muhammad Asim Shahid, Muhammad Mansoor Alam, Mazliham Mohd Su'ud | Applied Sciences | 2023 |

Table 12: The 67 extracted simulators from which the 5 simulators discussed in this study were selected. The simulators suggested by the research practitioner are labelled as having been extracted from *Manual*.

| | | | | | | |
|---|---|---|---|---|---|---|
| *OpenDC* | - | 2021 | 2025 | Manual | ✓ | ✓ |
| *NetworkCloudSim* | CloudSim | 2011 | 2025 | P1, P2, P3, P4, P5, P6, P7, P10, Manual | ✓ | ✓ |
| *CloudSim* | GridSim | 2009 | 2025 | P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, Manual | ✓ | ✓ |
| *SimGrid* | - | 2008 | 2025 | P3, P12 | ✓ | ✓ |
| *IoTSim-Stream* | CloudSim | 2019 | 2024 | P5 | ✓ | ✓ |
| *Ds-Sim* | - | 2019 | 2024 | P10 | ✓ | ✗ |
| *CloudSim Plus* | CloudSim | 2017 | 2024 | P13, Manual | ✓ | ✓ |
| *OCT (Open Cloud Testbed)* | - | 2009 | 2024 | P4 | ✓ | ✗ |
| *WCSim* | - | 2023 | - | Manual | ✗ | ✗ |
| *DISSECT-CF* | - | 2015 | 2023 | P1, P3, P5, P8, P11, P12 | ✓ | ✓ |
| *SCORE* | Google Omega simulator | 2018 | 2022 | P1 | ✓ | ✓ |
| *GPUCloudSim* | CloudSim | 2018 | 2022 | P5 | ✓ | ✓ |
| *DCworms* | GSSIM | 2013 | 2022 | P5 P8 | ✓ | ✓ |
| *iFogSim* | iCloudSim | 2017 | 2021 | P3, P5, P12 | ✓ | ✓ |
| *BigDataSDNSim* | CloudSimSDN, IoTSim | 2020 | 2020 | P5 | ✓ | ✓ |
| *CloudSimSDN* | CloudSim | 2015 | 2020 | P3, P5, P12 | ✓ | ✓ |
| *VMPlaces* | - | 2015 | 2020 | P5 | ✓ | ✗ |
| *DFaaSCloud* | CloudSim | 2019 | 2019 | P5 | ✓ | ✓ |
| *CactoSim* | Palladio, SimuLizar | 2014 | 2019 | P3 | ✓ | ✓ |
| *GAME-SCORE* | SCORE | 2018 | 2019 | P1, P11 | ✓ | ✓ |
| *NutShell* | NS3 | 2019 | - | P5 | ✗ | ✗ |
| *CloudSimDisk* | CloudSim | 2015 | 2018 | P3 P5 | ✓ | ✓ |
| *FederatedCloudSim* | CloudSim | 2014 | 2017 | P1 | ✓ | ✓ |
| *CloudReports* | CloudSim | 2014 | 2017 | P1, P3, P5, P6 | ✓ | ✓ |
| *ElasticSim* | CloudSim | 2017 | 2016 | P1, P6 | ✓ | ✓ |
| *CM Cloud* | CloudSim | 2016 | 2016 | P1 | ✓ | ✓ |
| *CloudAuction* | CloudSim | 2015 | 2016 | P12 | ✓ | ✓ |
| *Bazaar-Extension* | CloudSim | 2016 | - | P3, P5 | ✗ | ✓ |
| *ATAC4Cloud* | CloudSim | 2016 | - | P5 | ✗ | ✓ |
| *CloudSimEx* | CloudSim | 2012 | 2015 | P12 | ✓ | ✓ |
| *CEPSim* | CloudSim | 2015 | 2015 | P1, P3, P5, P6 | ✓ | ✓ |
| *DesktopCloudSim* | CloudSim | 2015 | 2015 | P5 | ✓ | ✓ |
| *DynamicCloudSim* | CloudSim | 2014 | 2015 | P1, P5 | ✓ | ✓ |
| *GloudSim* | - | 2014 | 2015 | P5 | ✓ | ✗ |
| *CloudSched* | - | 2013 | 2015 | P1, P3, P5, P6, P8 | ✓ | ✓ |
| *WorkflowSim* | CloudSim | 2012 | 2015 | P1, P3, P5, P6 | ✓ | ✓ |
| *iCanCloud* | SIMCAN | 2012 | 2015 | P1, P2, P3, P4, P5, P6, P7, P9, Manual | ✓ | ✓ |
| *SimGrid VM* | - | 2015 | - | P5 | ✗ | ✗ |

Table 12: Continued

| Name | Based on | First Published | Last Update | Extracted from | Code available | Energy simulation |
|---|---|---|---|---|---|---|
| *CloudExp* | CloudSim | 2014 | 2014 | P1, P3, P5, P6 | ✓ | ✓ |
| *CloudNetSim++* | OMNeT++ | 2014 | 2014 | P1, P3, P8 | ✓ | ✓ |
| *Cloud2Sim* | CloudSim, Hazelcast, Infinispan | 2014 | 2014 | P3, P5 | ✓ | ✓ |
| *CloudSimScale* | CloudSim | 2014 | 2014 | P5 | ✓ | ✓ |
| *TeachCloud* | CloudSim | 2013 | 2014 | P1, P3, P6, P7 | ✓ | ✓ |
| *CRest* | - | 2013 | 2014 | P5 | ✓ | ✓ |
| *DCSim* | - | 2012 | 2014 | P2, P3, P4, P5, P6, P8, P10 | ✓ | ✓ |
| *secCloudSim* | iCanCloud | 2014 | - | P1, P6 | ✗ | ✗ |
| *GreenCloud* | NS2 | 2010 | 2013 | P1, P2, P3, P4, P6, P7, P8, P9, P10, P11, P12, Manual | ✓ | ✓ |
| *CloudNetSim* | - | 2013 | - | P1 | ✗ | ✗ |
| *SimIC* | - | 2013 | - | P1, P2, P5, P6, P10 | ✗ | ✓ |
| *E-mc2* | iCanCloud | 2013 | - | P5, P8 | ✗ | ✓ |
| *SGCB +(SimGrid Cloud Broker)* | SimGrid | 2013 | - | P5 | ✗ | ✗ |
| *DartCSim+* | CloudSim | 2013 | - | P1 | ✗ | ✓ |
| *FTCloudSim* | CloudSim | 2013 | - | P1, P6, P9 | ✗ | ✓ |
| *EMUSim* | AEF, CloudSim | 2012 | 2012 | P1, P2, P3, P4, P5, P6, P9 | ✓ | ✓ |
| *BigHouse* | - | 2012 | 2012 | P7, P8 | ✓ | ✓ |
| *SPECI* | - | 2009 | 2012 | P1, P3, P4, P5, P6, P7 | ✗ | ✗ |
| *MR-CloudSim* | CloudSim | 2012 | - | P1, P2, P3, P6 | ✗ | ✓ |
| *UCloud* | CloudSim | 2012 | - | P1, P3 | ✗ | ✓ |
| *SmartSim* | - | 2012 | - | P2, P5 | ✗ | ✓ |
| *CDOSim* | CloudSim | 2012 | - | P1, P2, P3, P7 | ✗ | ✓ |
| *DartCSim* | CloudSim | 2012 | - | P1, P3, P6 | ✗ | ✓ |
| *CloudAnalyst* | CloudSim | 2009 | 2010 | P1, P2, P3, P4, P5, P6, P7, P9, P10, P12, P13, Manual | ✓ | ✓ |
| *GDCSim* | - | 2011 | - | P1, P3, P5, P7, P8, P11 | ✗ | ✓ |
| *GroudSim* | - | 2011 | - | P1, P2, P3, P4, P5, P6, P7, P10, Manual | ✗ | ✗ |
| *MDCSim* | CSIM | 2009 | - | P1, P2, P3, P5, P6, P7, P8, P10 | ✗ | ✓ |
| *Open Cirrus* | - | 2009 | - | P4 | ✗ | ✗ |
| *DGSim* | - | 2008 | - | Manual | ✗ | ✗ |

## 12. Simulator Output Screenshots

### *12.1. CloudSim and NetworkCloudSim*

```
Power data center: New resource usage for the time frame starting at 301.02:

301.02: [Host #0] utilization is 0.000000%

Energy consumption for the last time frame from 300.02 to 301.02:

301.02: [Host #0] utilization at 300.02 was 81.25%, now is 0.00%
301.02: [Host #0] energy is 2.67 W*sec

301.02: Data center's energy is 2.67 W*sec

The number of Containers Up and running is 0
The number of Vms Up and running is 0
The number of Hosts Up and running is 0
Over-utilized hosts:

Reallocation of VMs from the over-utilized hosts:

301.02: The number of Migrations is:  0
301.02: Simulation: No more future events
301.02: CloudInformationService is shutting down...
301.02: CloudInformationService: Notify all CloudSim entities for shutting down.
301.02: Broker is shutting down...
301.02: Datacenter is shutting down...
Simulation completed.
Simulation completed.

========== OUTPUT ==========
Cloudlet ID    STATUS    Data center ID    Vm ID    Time    Start Time    Finish Time
    0        SUCCESS        3            0        300        0.02        300.02
    1        SUCCESS        3            1        300        0.02        300.02
    2        SUCCESS        3            2        300        0.02        300.02
    3        SUCCESS        3            3        300        0.02        300.02
    4        SUCCESS        3            4        300        0.02        300.02
    5        SUCCESS        3            5        300        0.02        300.02
    6        SUCCESS        3            6        300        0.02        300.02
    7        SUCCESS        3            7        300        0.02        300.02
    8        SUCCESS        3            8        300        0.02        300.02
    9        SUCCESS        3            9        300        0.02        300.02
    10        SUCCESS        3            10        300        0.02        300.02
    11        SUCCESS        3            11        300        0.02        300.02
    12        SUCCESS        3            12        300        0.02        300.02
datacenter power: 1597.7203069372542 Joules
TraceExperiment finished!
```

Figure 10: Screenshot of the end of terminal output of an example *CloudSim* or *NetworkCloudSim* simulation.

### *12.2. DISSECT-CF*

```
Done setting up Taxonomy
Standard workload format trace file reader starts for: ../../../traces/sockshop/A/100/18/trace.swf at Sun Jul 06 21:13:33 CEST 2025
Standard workload format trace file reader stops for: ../../../traces/sockshop/A/100/18/trace.swf at Sun Jul 06 21:13:33 CEST 2025
Job dispatcher (with 780 jobs) is completely prepared at 1751829213525
Current simulation time: 5
Power metering started with delay 1
VM startup timeout is set to 86400000
Total power consumption: 2056.8824283591803 Joules
Simulation terminated 1751829214098 (took 573ms in realtime)
Current simulation time: 307
Simulated timespan: 302 seconds
Final number of: Ignored jobs - 0 Destroyed VMs - 780 reused VMs: 0
Average number of PMs in running state: 0.9966777408637874
Maximum number of running PMs during the whole simulation: 1
Performance: 1.361256544502618 VMs/ms
```

Figure 11: Screenshot of terminal output of an example *DISSECT-CF* simulation.

### *12.3. OpenDC*

```
20:59:23.658 [WARN ] org.opendc.compute.simulator.telemetry.ComputeMetricReader -
                                        Metrics after 0 hours:
                                                Tasks Total: 69
                                                Tasks Active: 2
                                                Tasks Pending: 0
                                                Tasks Completed: 67
                                                Tasks Terminated: 0
Simulating... 100% [==================================] 8/8 (0:00:03 / 0:00:00)
```

Figure 12: Screenshot of the end of terminal output of an example *OpenDC* simulation.

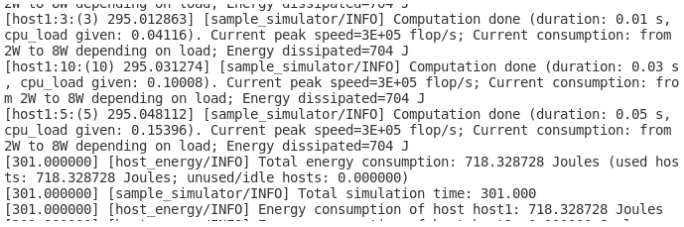



Figure 13: Screenshot of the end of terminal output of an example *SimGrid* simulation.